2110521 Software Architecture — Section WED (วันพุธ)

Assignment 05
Microservice API Gateway using KONG

ติดตั้งและใช้งาน KONG เป็น API Gateway หน้า HTTP / REST / gRPC services
พร้อม Konga (UI), Upstream load balancer, Basic Auth และ Key Auth

ชื่อ	Watayut Aiamanan
รหัสนิสิต	6970267821
วันที่ส่ง	20 **ตุลาคม** 2026
IP address **ของเครื่อง**	10.0.10.51 (**แทน** localhost **ตามที่โจทย์กำหนด**)

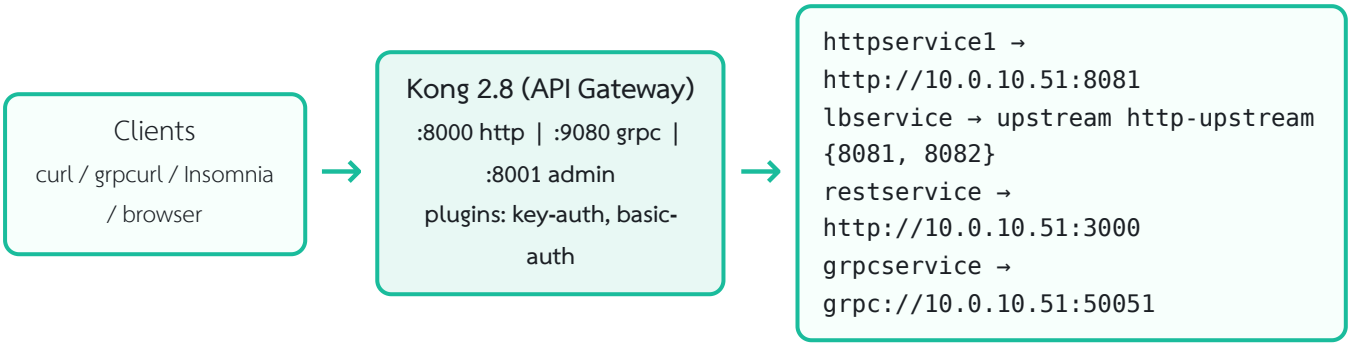
> **โจทย์:** ติดตั้งและใช้งาน KONG เป็น API Gateway เพื่อไปใช้กับ REST/gRPC Services และ capture หน้าจอแสดง services และ routes ที่ใช้ (กรณีที่ Services อยู่ที่ localhost ให้ใช้ ip address ของตนแทน)

1. ภาพรวมและสภาพแวดล้อมที่ใช้

งานนี้ทำบน GitHub Codespaces (Ubuntu 24.04, Docker-in-Docker) แทน Windows 10 ตาม tutorial จึงใช้คำสั่ง `hostname -I` แทน `ipconfig` ในการหา IP address ของเครื่อง ซึ่งได้ 10.0.10.51 และใช้ IP นี้เป็น host ของทุก service ที่ลงทะเบียนใน Kong

ส่วนประกอบ	เวอร์ชัน / ค่าที่ใช้	หน้าที่
Docker	29.8.0	รัน Postgres, Kong, Konga เป็น container ใน network `kong-net`
PostgreSQL	**postgres:9.6** (ตาม PDF)	ฐานข้อมูลของ Kong และ Konga
Kong	**kong:2.8** (2.8.5)	API Gateway: proxy :8000, admin :8001, gRPC proxy :9080
Konga	pantsel/konga 0.14.9	Web UI สำหรับจัดการ Kong ที่ :1337
Node.js	v24.21.0	รัน SimpleHTTP (8081, 8082), SimpleREST (3000), SimplegRPC (50051)
Insomnia	13.3.1 (Linux AppImage บน Xvfb)	ยิงทดสอบ HTTP / REST / gRPC ผ่าน Kong
curl / grpcurl	grpcurl 1.9.4	ทดสอบจาก terminal

หมายเหตุเรื่องเวอร์ชัน: PDF ระบุ `docker pull kong` (latest) ซึ่งในปี 2022 คือ Kong 2.x แต่ปัจจุบัน latest คือ Kong 3.x ที่ Konga รองรับไม่สมบูรณ์ จึงเลือก `kong:2.8` คู่กับ `postgres:9.6` ตาม PDF เพื่อให้ Konga ทำงานได้ครบ



Konga 0.14.9 (:1337) → Kong Admin API (http://kong:8001) Kong / Konga → PostgreSQL 9.6 (kong-database)

2. Step 0-1: ติดตั้ง Docker, Node.js และสร้าง container ของ Postgres 9.6 และ Kong

```
codespace@codespace_launcher: bash — 01-docker-version

$ docker -v && node -v && npm -v
Docker version 29.8.0-1, build 88096ef00576baf72a9cb45caa45c0544c40e0a7
v24.21.0
11.19.0
[exit 0]
```

ตรวจสอบเวอร์ชัน Docker, Node.js และ npm

```
codespace@codespace_launcher: bash — 02-docker-images

$ docker images | grep -E 'REPOSITORY|postgres|kong|konga'
kong:2.8              34bbeaf6ee4c        151MB         36.5MB
pantsel/konga:latest  c8172b75607d        776MB          146MB
postgres:9.6          caddd35b05cd        294MB         73.6MB
[exit 0]
```

image ที่ pull มา: postgres:9.6, kong:2.8, pantsel/konga

```
codespace@codespace_launcher: bash — 03-network-create

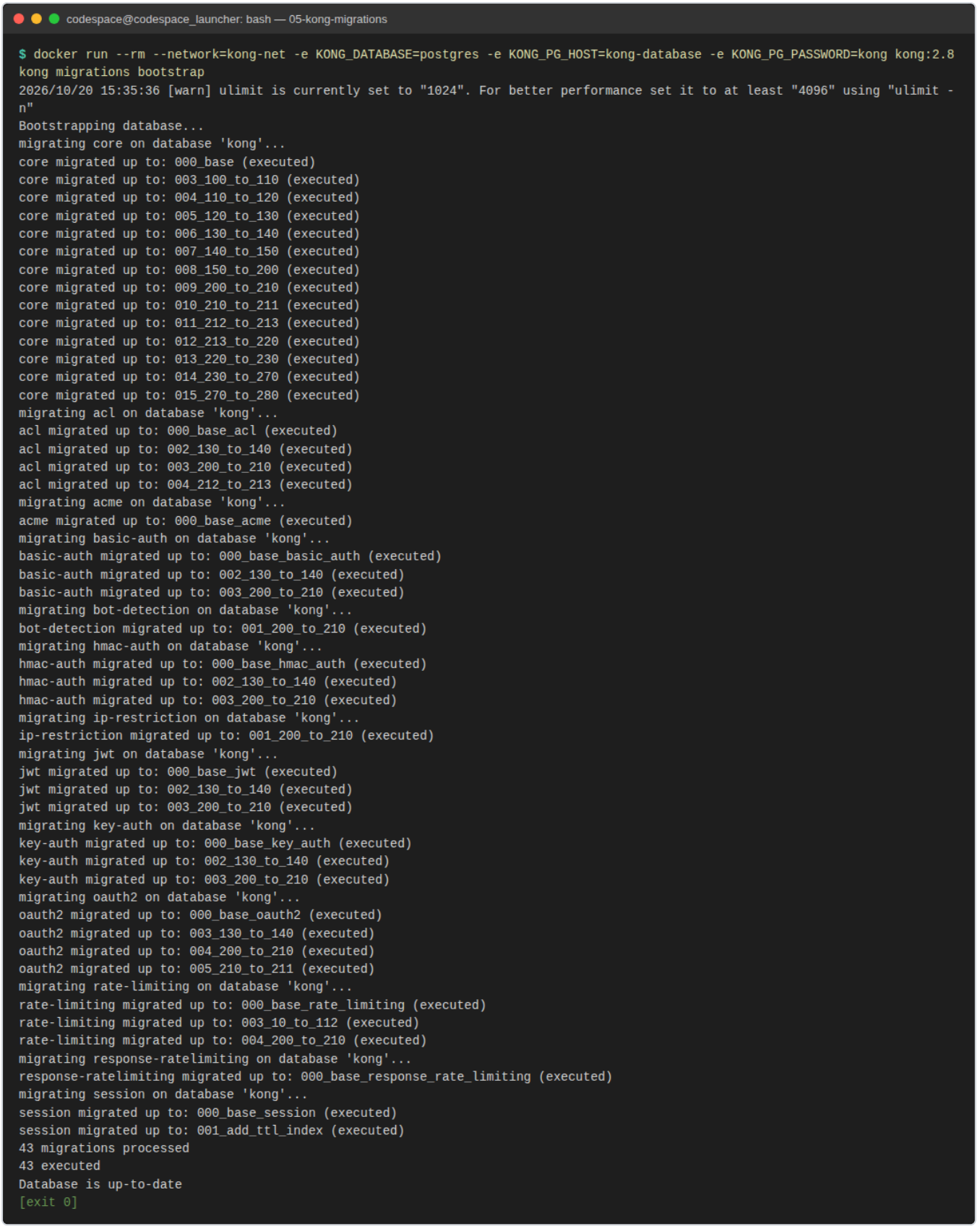
$ docker network create kong-net
4be2d4435d3725222eff98aa7d17faf1291c2b8afedd8566e18a28d41dccaed9
[exit 0]
```

1.3 a) สร้าง docker network ชื่อ kong-net

```
codespace@codespace_launcher: bash — 04-run-postgres

$ docker run -d --name kong-database --network=kong-net -p 5432:5432 -e POSTGRES_USER=kong -e POSTGRES_DB=kong -e POSTGRES_PAS
SWORD=kong postgres:9.6
a78fe6d47fae5ef4cc9e313061324ad0ef5affc2b127af7176ebad080fd4d55f
[exit 0]
```

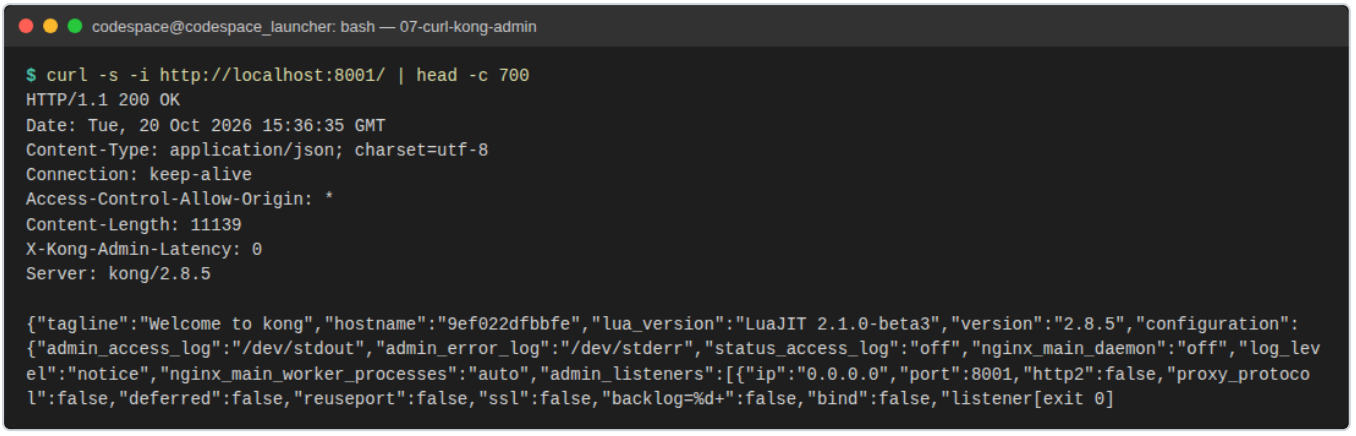
1.3 b) รัน Postgres 9.6 ชื่อ kong-database ใน kong-net



1.3 c) **รัน** kong migrations bootstrap **เพื่อเตรียมฐานข้อมูลให้** Kong



1.3 d) รัน Kong พร้อมเปิด admin :8001 และ proxy :8000 / :9080 (http2 สำหรับ gRPC) ตามคำสั่งใน PDF



1.3 e) ตรวจสอบว่า Kong ทำงาน: curl http://localhost:8001/ ได้ 200 OK, Server: kong/2.8.5

3. Step 2: ติดตั้ง Konga และเชื่อมต่อกับ Kong

```
codespace@codespace_launcher: bash — 08-konga-prepare

$ docker run --rm --network=kong-net pantsel/konga -c prepare -a postgres -u postgresql://kong:kong@kong-database:5432/konga
[34mdebug: [39mPreparing database...
Using postgres DB Adapter.
Database `konga` does not exist. Creating...
Database `konga` created! Continue...
[34mdebug: [39mHook:api_health_checks:process() called
[34mdebug: [39mHook:health_checks:process() called
[34mdebug: [39mHook:start-scheduled-snapshots:process() called
[34mdebug: [39mHook:upstream_health_checks:process() called
[34mdebug: [39mHook:user_events_hook:process() called
[34mdebug: [39mSeeding User...
[34mdebug: [39mUser seed planted
[34mdebug: [39mSeeding Kongnode...
[34mdebug: [39mKongnode seed planted
[34mdebug: [39mSeeding Emailtransport...
[34mdebug: [39mEmailtransport seed planted
[34mdebug: [39mDatabase migrations completed!
[exit 0]
```

2.2 a) เตรียมฐานข้อมูล konga ใน Postgres เดียวกัน (prepare)

```
codespace@codespace_launcher: bash — 09-run-konga

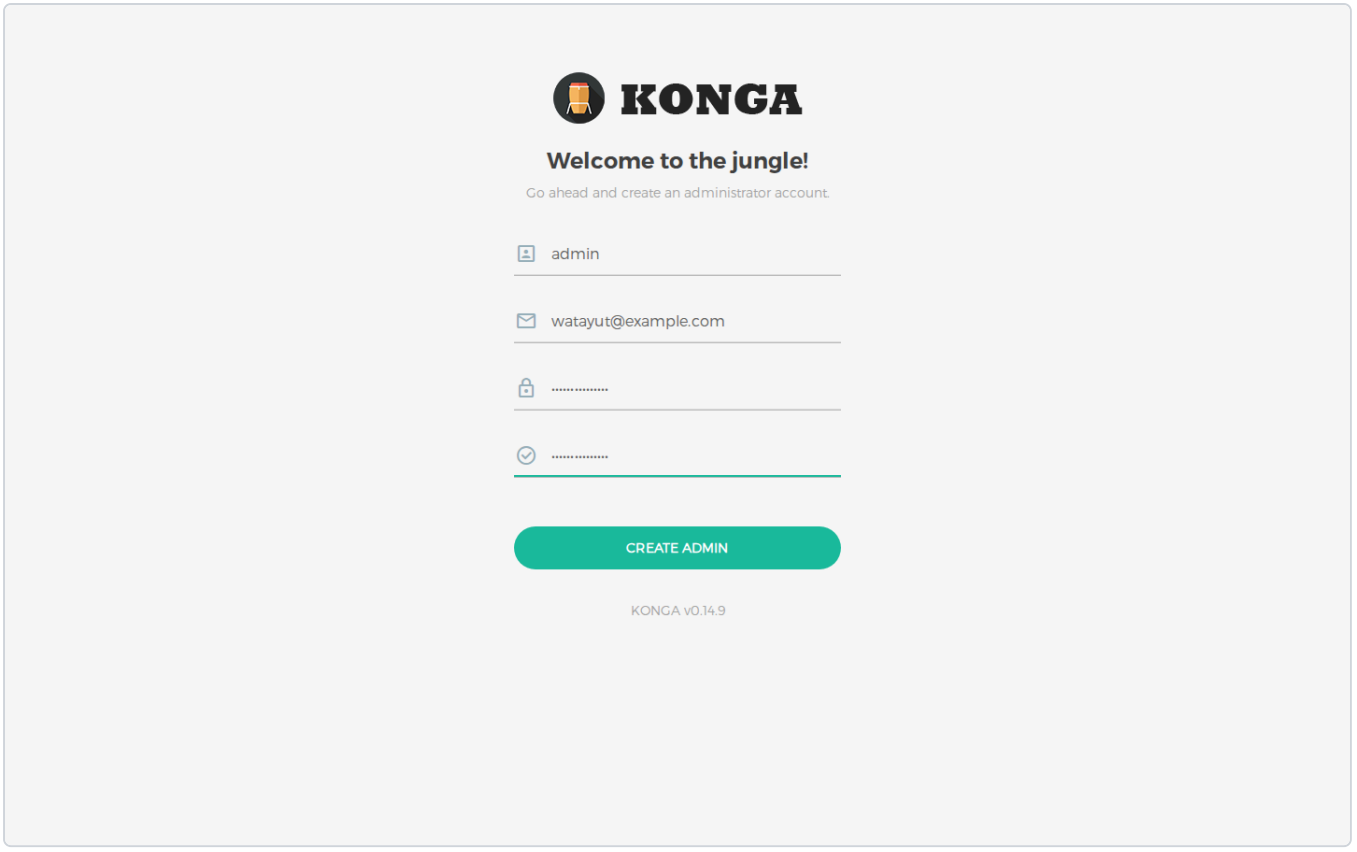
$ docker run -d -p 1337:1337 --network=kong-net -e DB_ADAPTER=postgres -e DB_HOST=kong-database -e DB_URI=postgresql://kong:ko
ng@kong-database:5432/konga -e KONGA_HOOK_TIMEOUT=120000 -e NODE_ENV=production --name konga pantsel/konga
a0ad70d291c46c08128914dc2d436e1a6d37ec6bef9a1da1abb00e0540cb347a
[exit 0]
```

2.2 b) รัน Konga ที่ port 1337

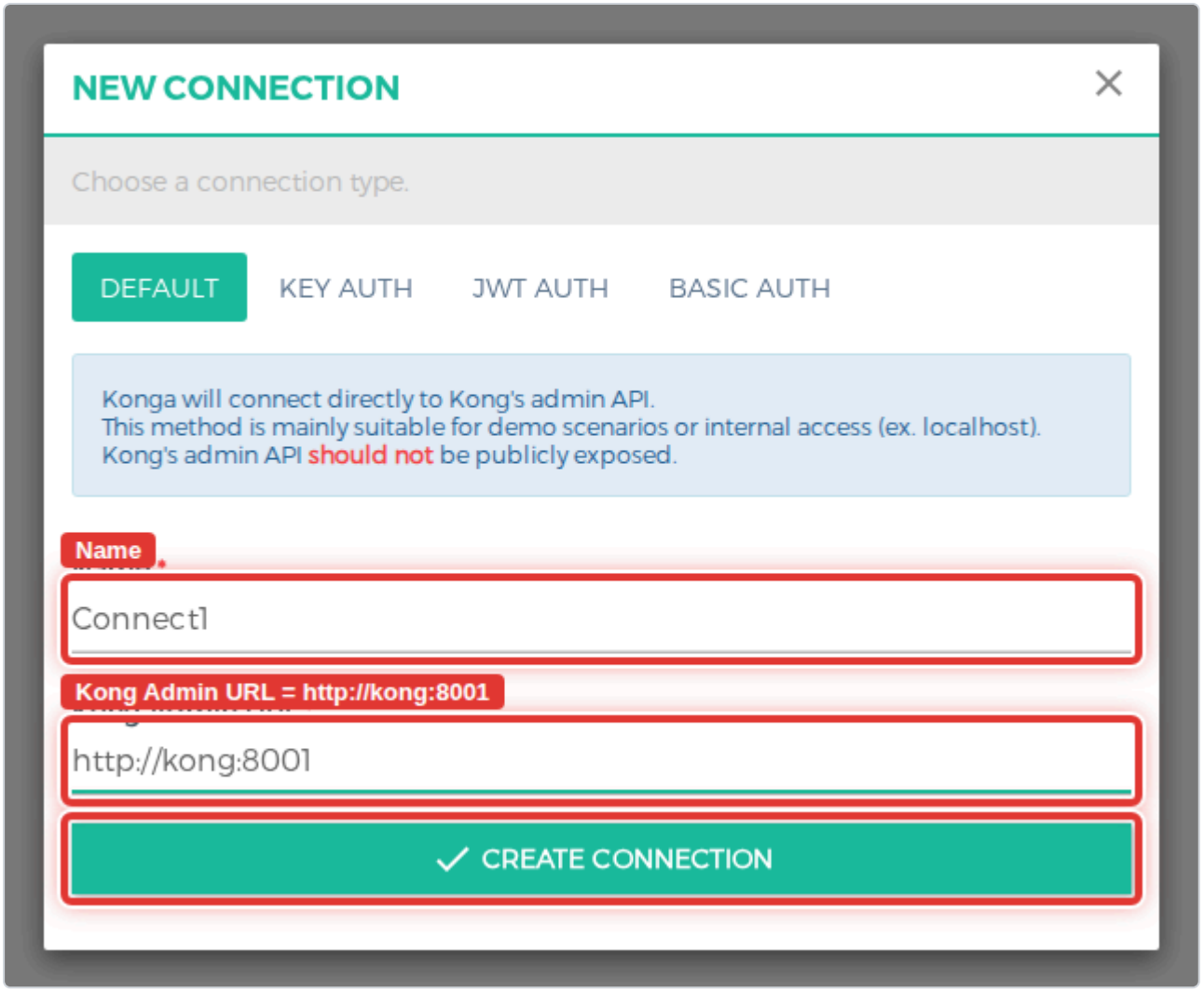
```
codespace@codespace_launcher: bash — 10-docker-ps

$ docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
NAMES           IMAGE           STATUS                            PORTS
konga           pantsel/konga   Up 15 seconds                     0.0.0.0:1337->1337/tcp, [::]:1337->1337/tcp
kong            kong:2.8        Up 38 seconds (health: starting)  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 127.0.0.1:8001
->8001/tcp, 127.0.0.1:8444->8444/tcp, 0.0.0.0:8443->8443/tcp, [::]:8443->8443/tcp, 127.0.0.1:9080-9081->9080-9081/tcp
kong-database   postgres:9.6    Up 4 minutes                      0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
[exit 0]
```

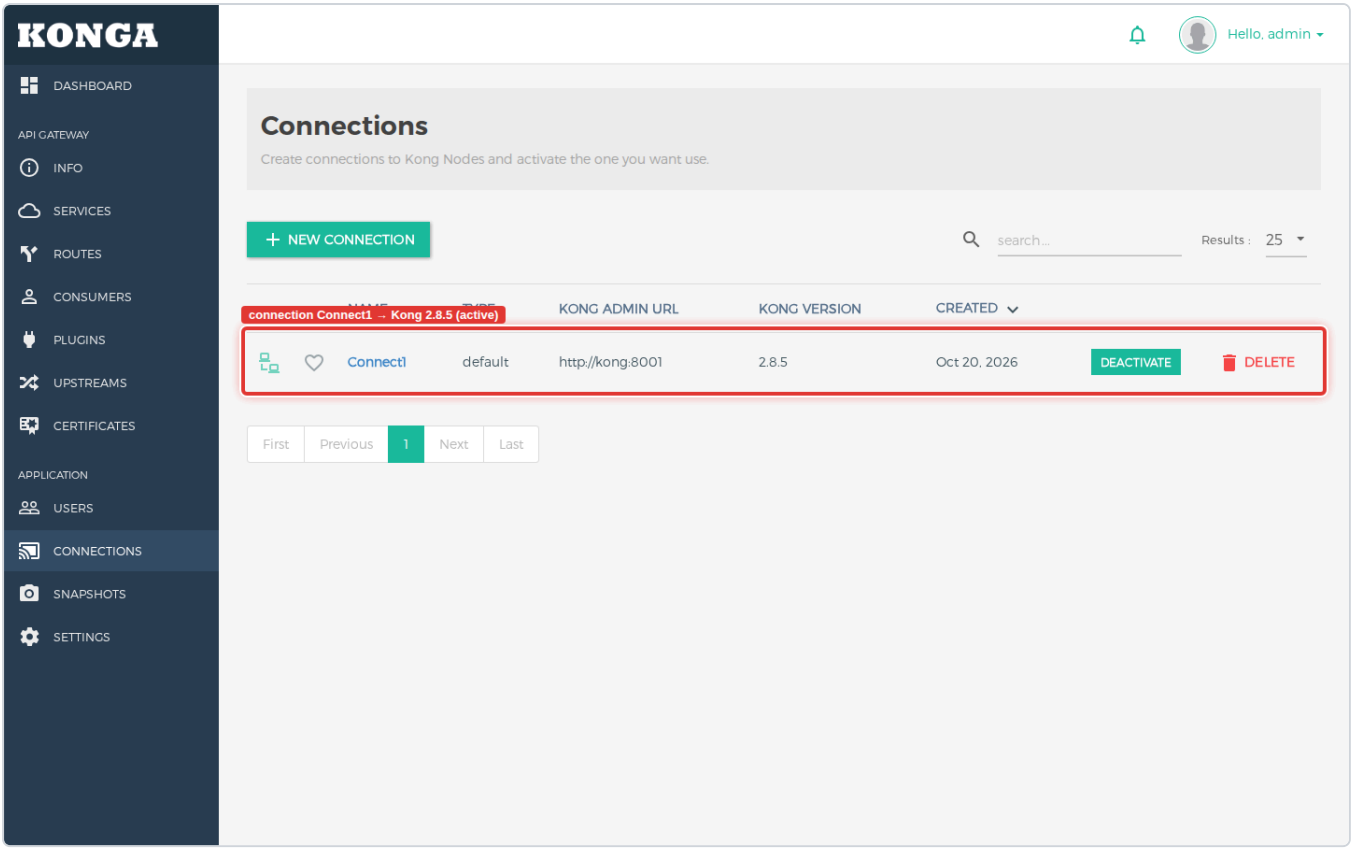
container ทั้ง 3 ตัวทำงานอยู่: konga, kong, kong-database



2.2 d) **สร้าง** admin user **ของ** Konga (http://localhost:1337)



2.2 e) สร้าง connection ชื่อ Connect1 ไปที่ Kong Admin URL = http://kong:8001



Konga เชื่อมต่อ Kong สำเร็จ แสดง Kong version 2.8.5 และสถานะ Connected to Connect1

4. Start HTTP / REST / gRPC services ด้วย Node.js (2.2 f)

start service ทั้ง 4 ตัวจาก source code ของอาจารย์ (SimpleServices) ค้างไว้ใน background แล้วตรวจสอบว่าเรียกตรง ๆ ผ่าน IP ของเครื่องได้

```
cd SimpleHTTP   && node main1.js            # Hello World from Service1  :8081
cd SimpleHTTP   && node main2.js            # Hello World from Service2  :8082
cd SimpleREST   && node app.js              # express REST /account      :3000
cd SimplegRPC/node && node greeter_server.js # helloworld.Greeter/SayHello :50051
```

```
codespace@codespace_launcher: bash — 11-ip-address

$ hostname -I | awk '{print $1}'
10.0.10.51
[exit 0]
```

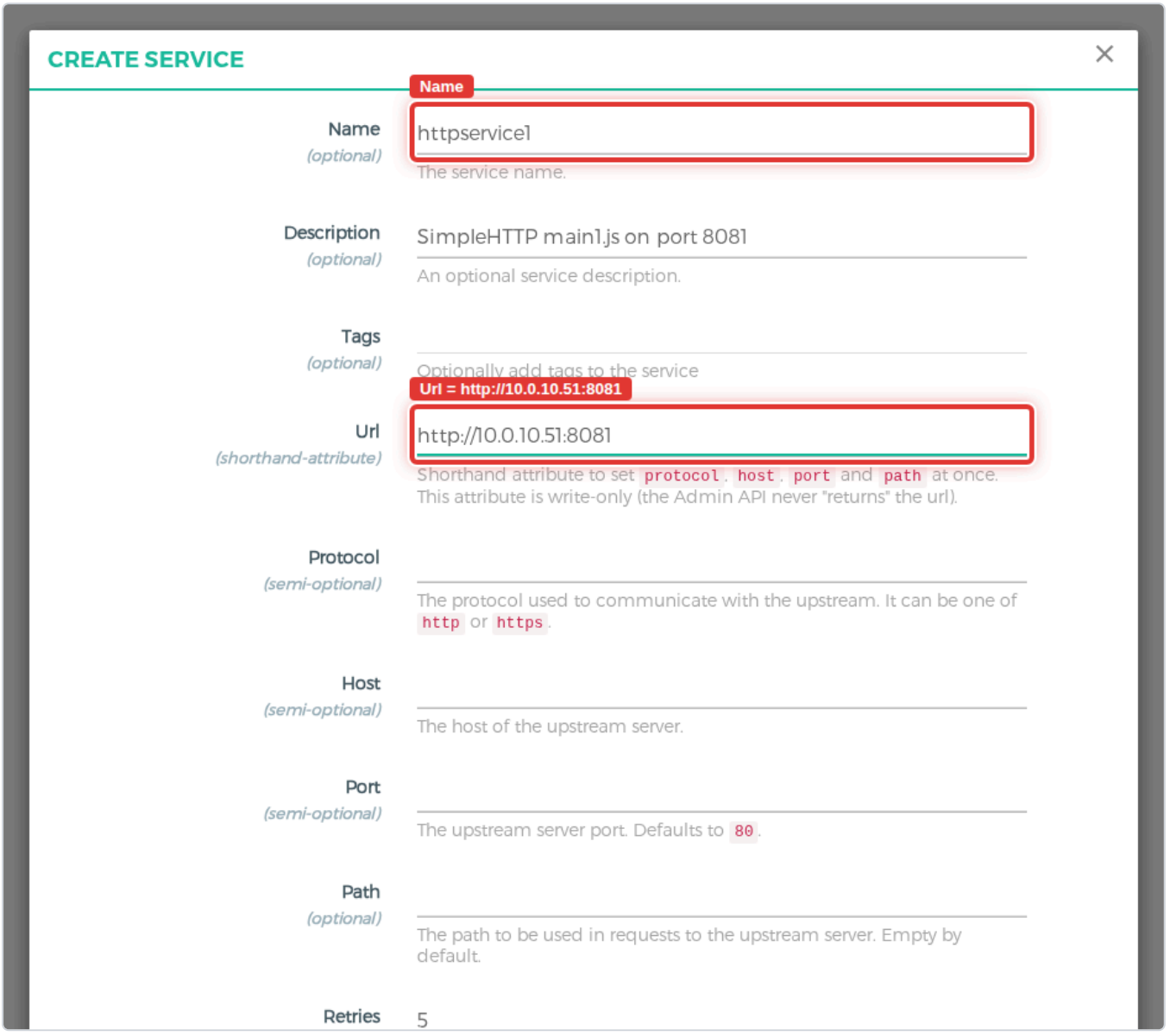
หา IP address ของเครื่อง (ใช้แทน localhost)

```
codespace@codespace_launcher: bash — 12-services-direct

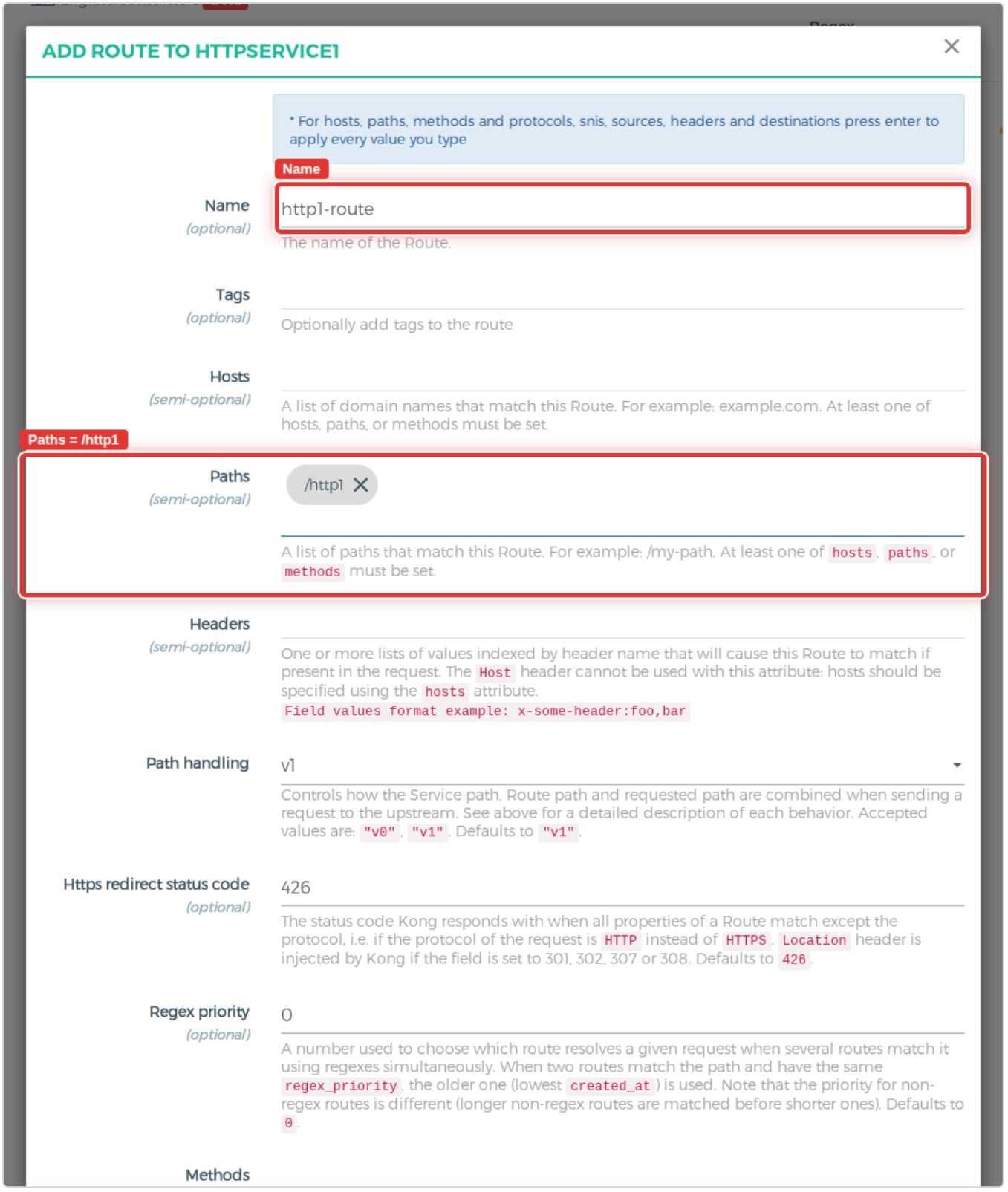
$ curl -s http://10.0.10.51:8081/; curl -s http://10.0.10.51:8082/; curl -s 'http://10.0.10.51:3000/account?username=nraboy';
echo
Hello World from Service1
Hello World from Service2
{"username":"nraboy","password":"1234","twitter":"@nraboy"}
[exit 0]
```

เรียก service ตรง ๆ ที่ IP ของเครื่อง ก่อนผ่าน Kong

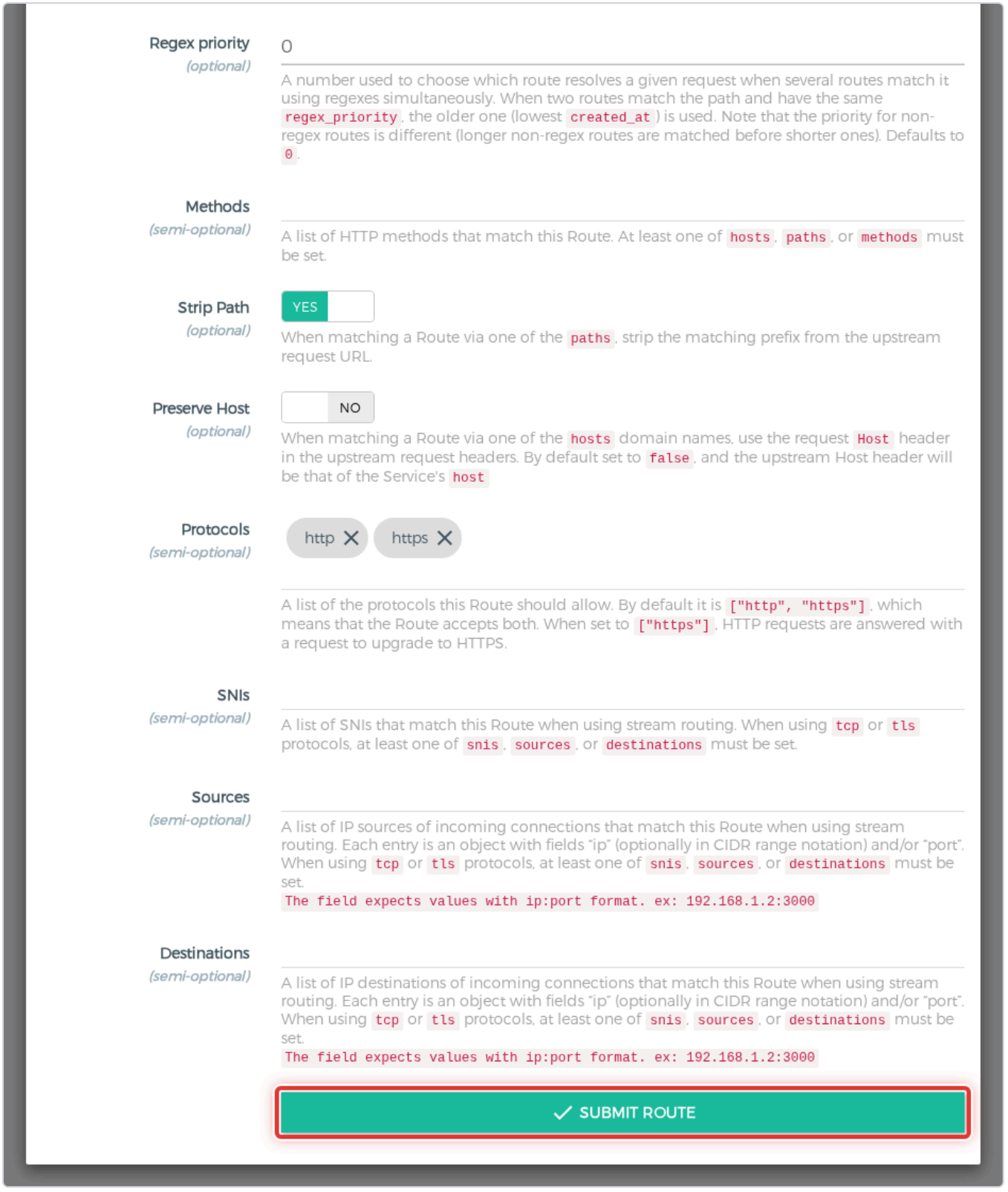
5. Step 3: **สร้าง** Service **และ** Route **สำหรับ** HTTP (httpservice1 → :8081) **ด้วย** Konga



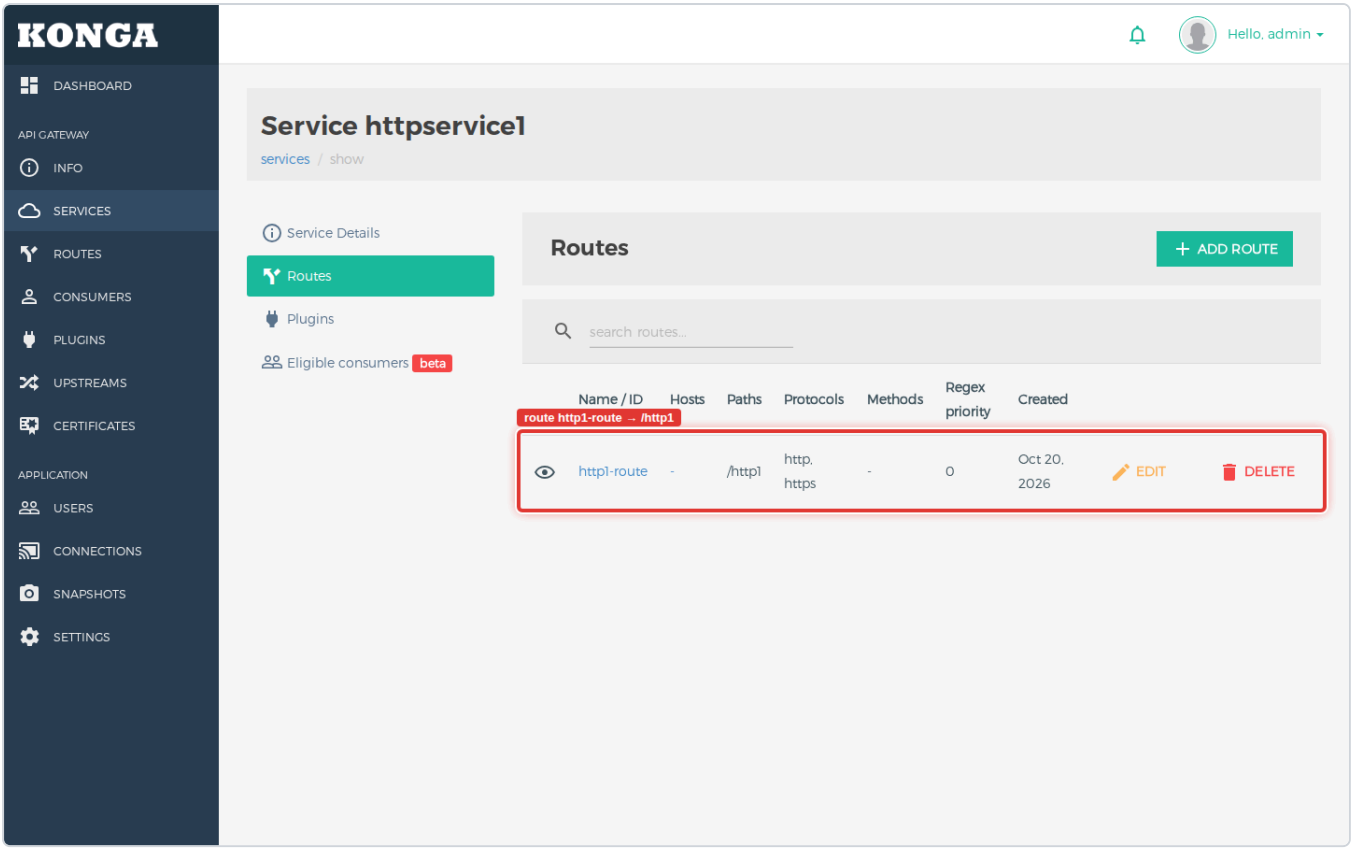
Add service: name = httpservice1, url = http://10.0.10.51:8081



Add route ให้ httpservice1: name = http1-route, paths = /http1 (ส่วนบน)



Add route ให้ httpservice1: name = http1-route, paths = /http1 (ส่วนล่าง)



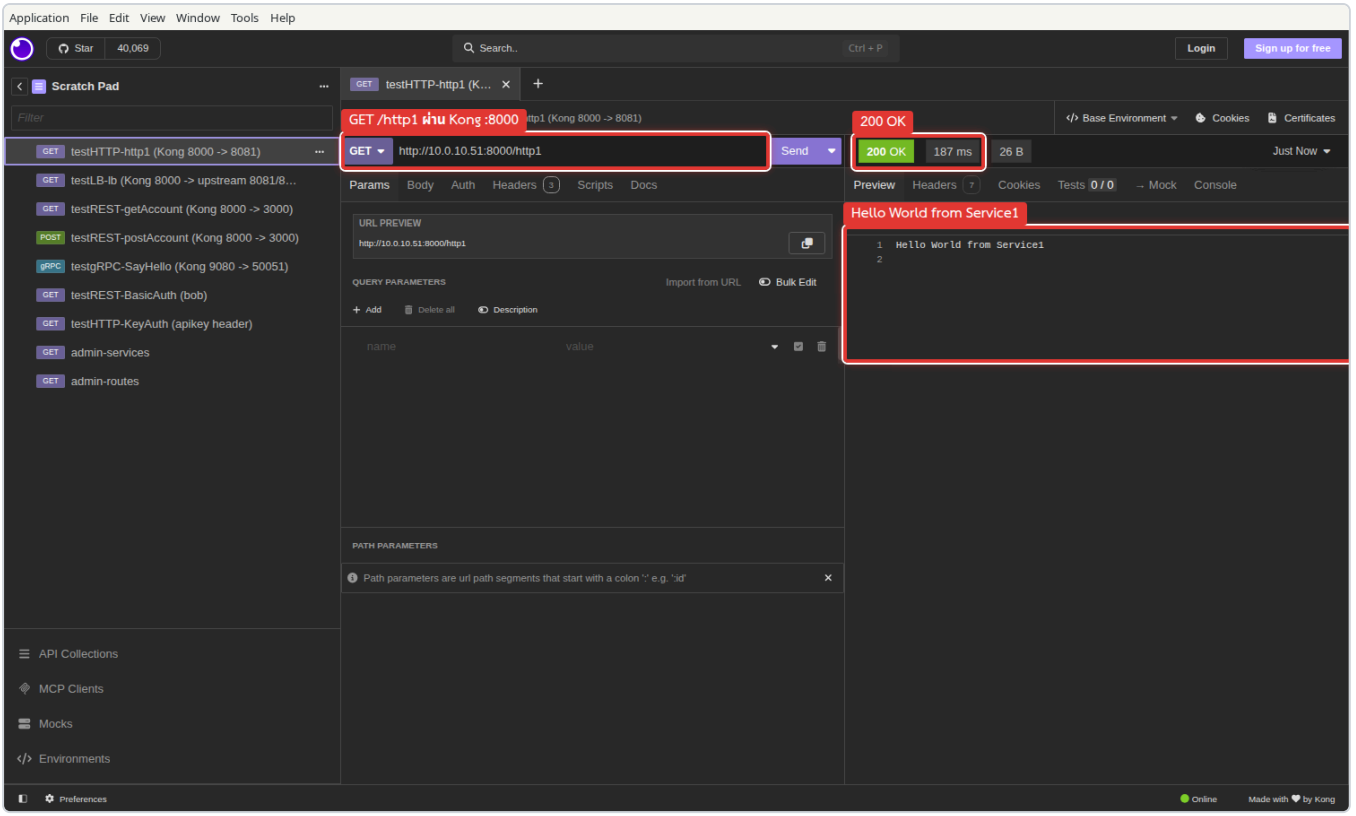
Route ของ httpservice1 ถูกสร้างแล้ว

```
codespace@codespace_launcher: bash — 16-test-http1-via-kong

$ curl -s -i http://10.0.10.51:8000/http1
HTTP/1.1 200 OK
Content-Type: text/plain; charset=UTF-8
Transfer-Encoding: chunked
Connection: keep-alive
Date: Tue, 20 Oct 2026 15:58:15 GMT
X-Kong-Upstream-Latency: 1
X-Kong-Proxy-Latency: 0
Via: kong/2.8.5

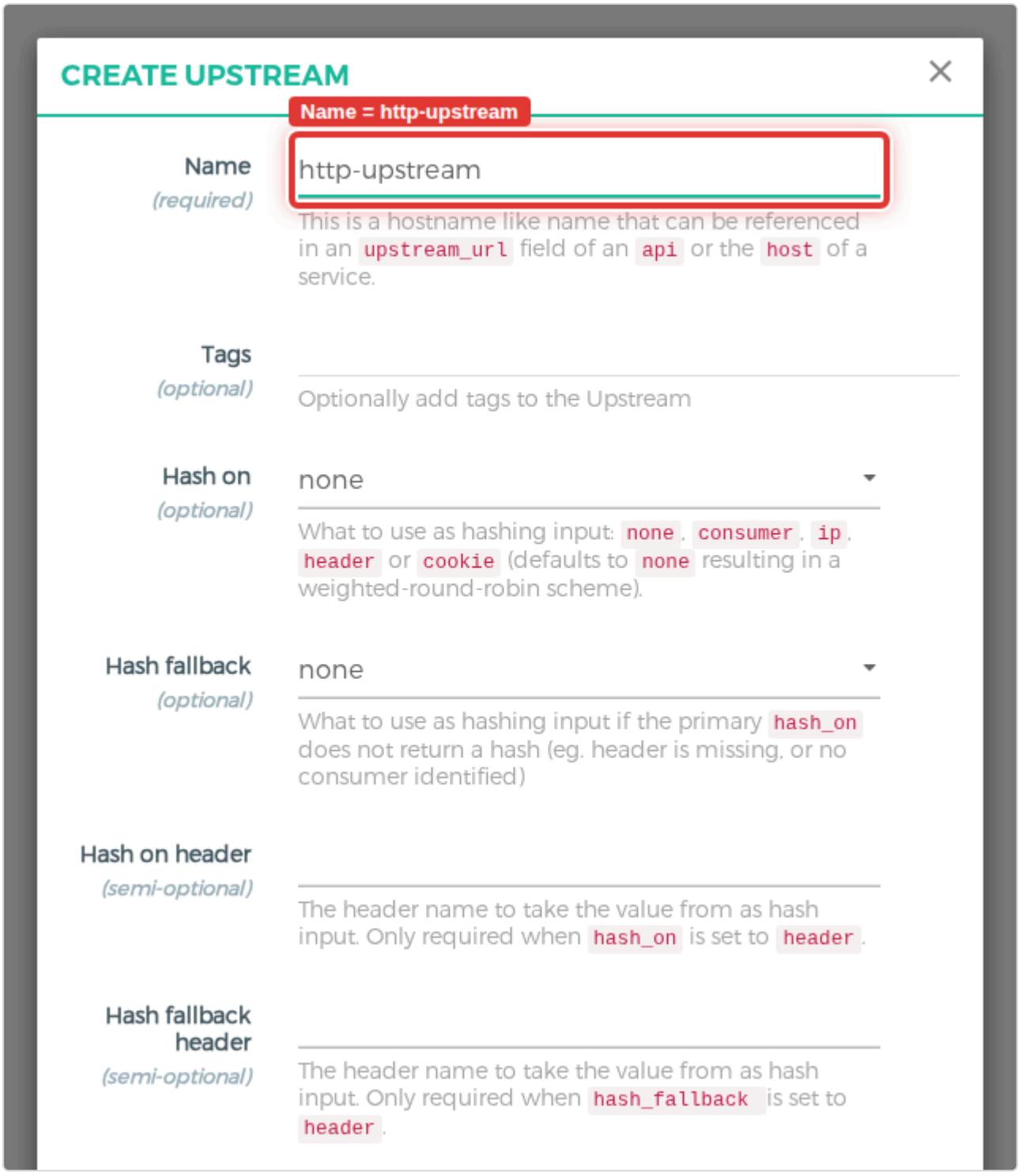
Hello World from Service1
[exit 0]
```

ทดสอบผ่าน Kong: GET http://10.0.10.51:8000/http1 → 200 OK ผ่าน Via: kong/2.8.5

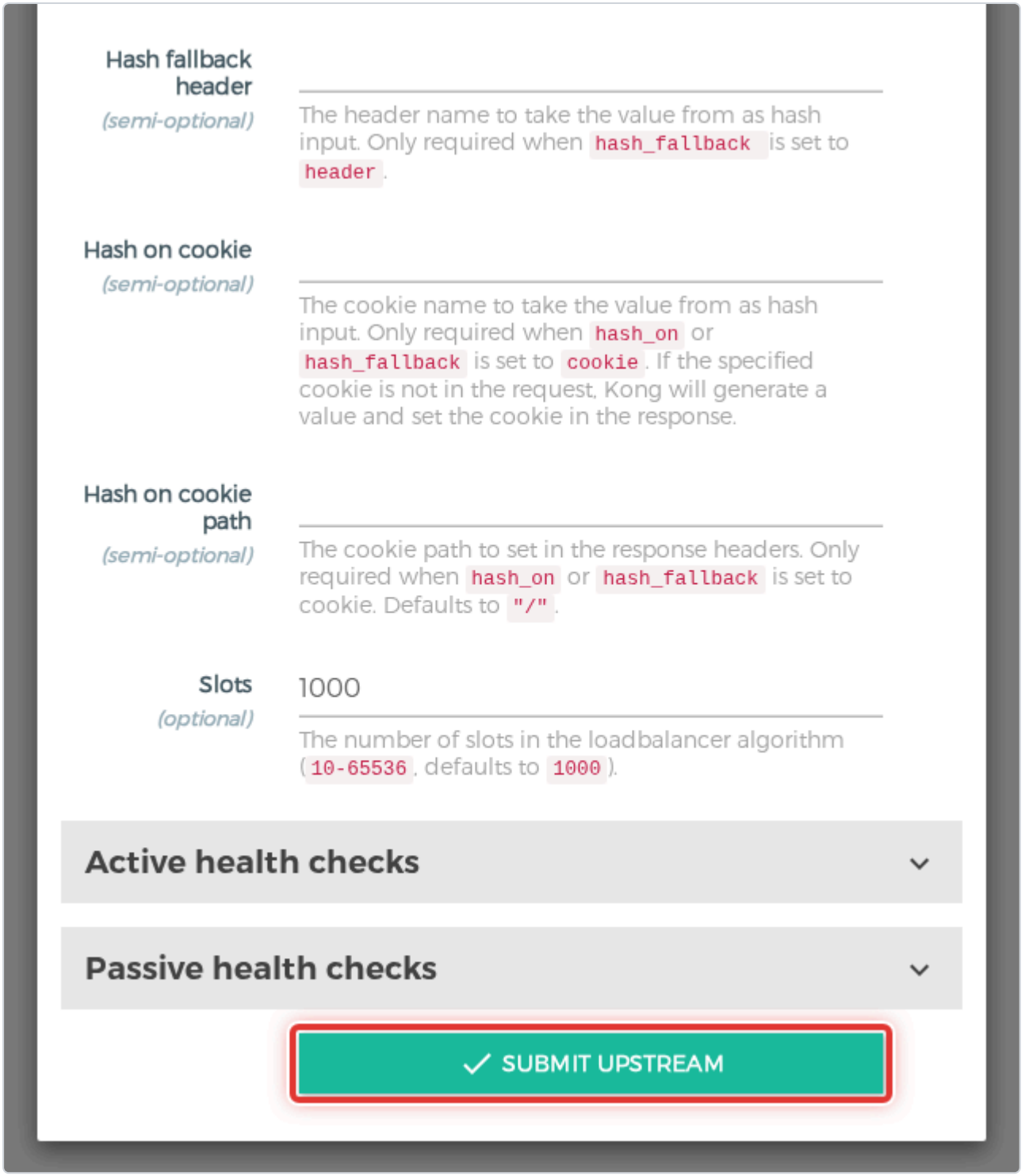


Insomnia: GET http://10.0.10.51:8000/http1 → 200 OK "Hello World from Service1"

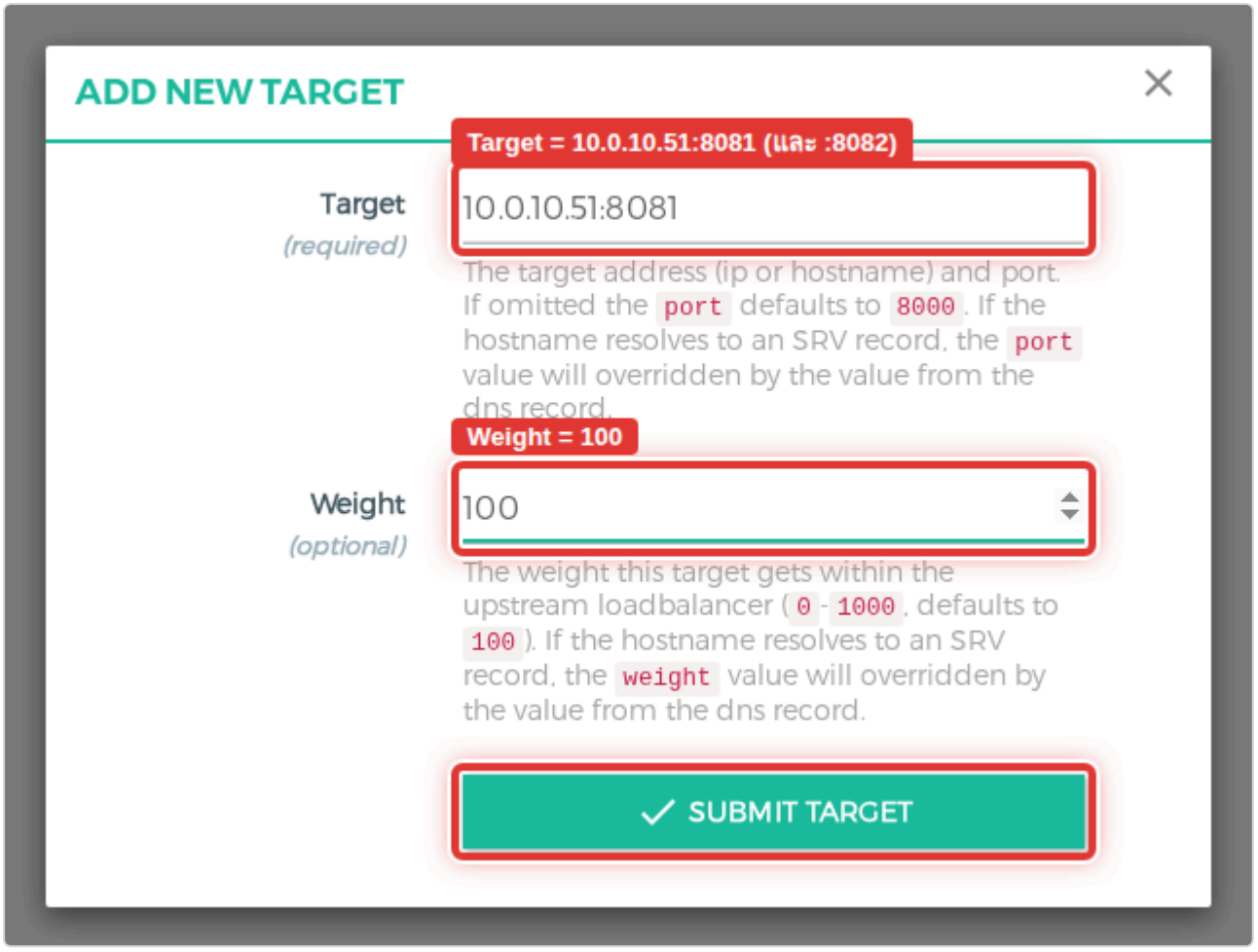
6. Step 4: Upstream (load balancer) ไปที่ :8081 และ :8082



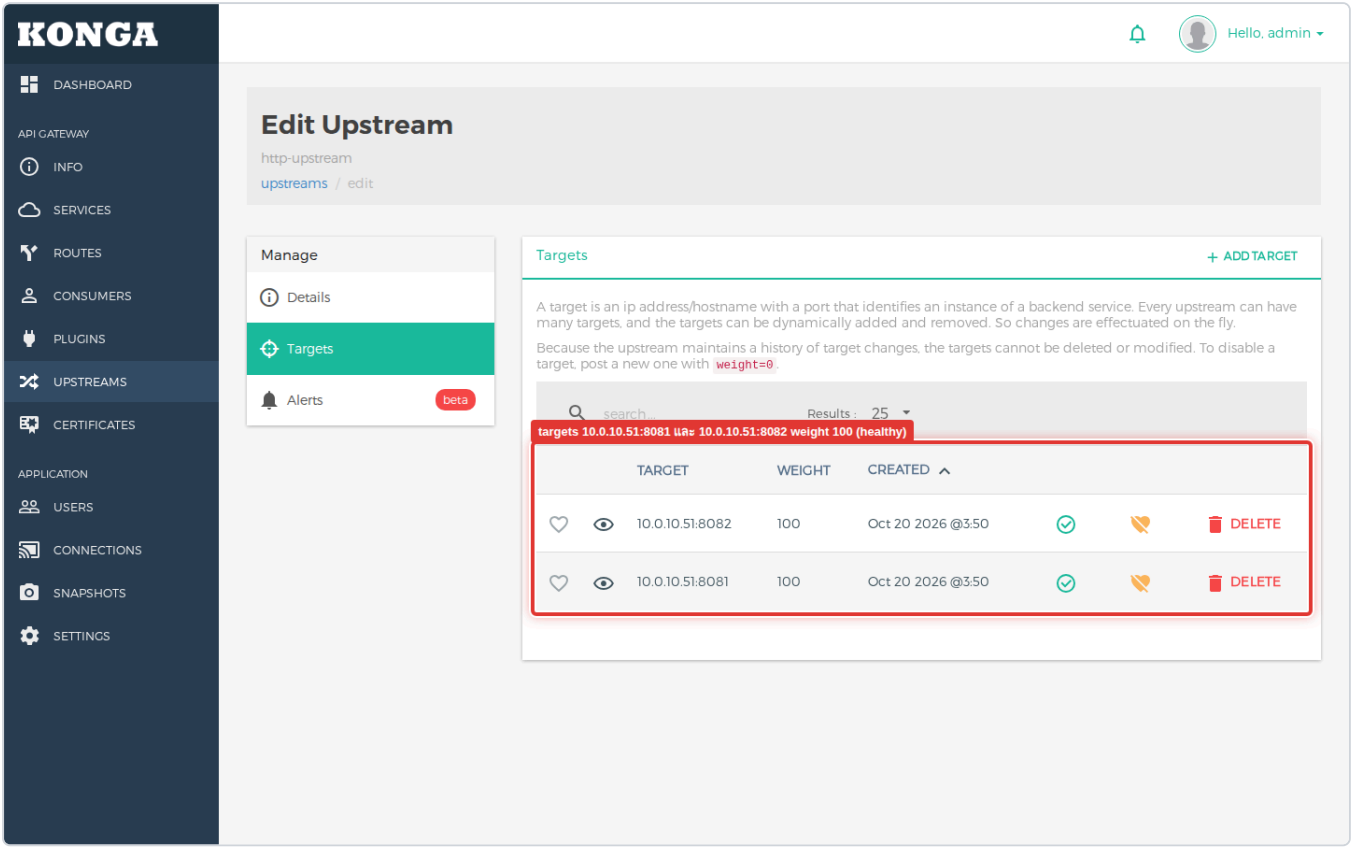
สร้าง upstream ชื่อ http-upstream (ส่วนบน)



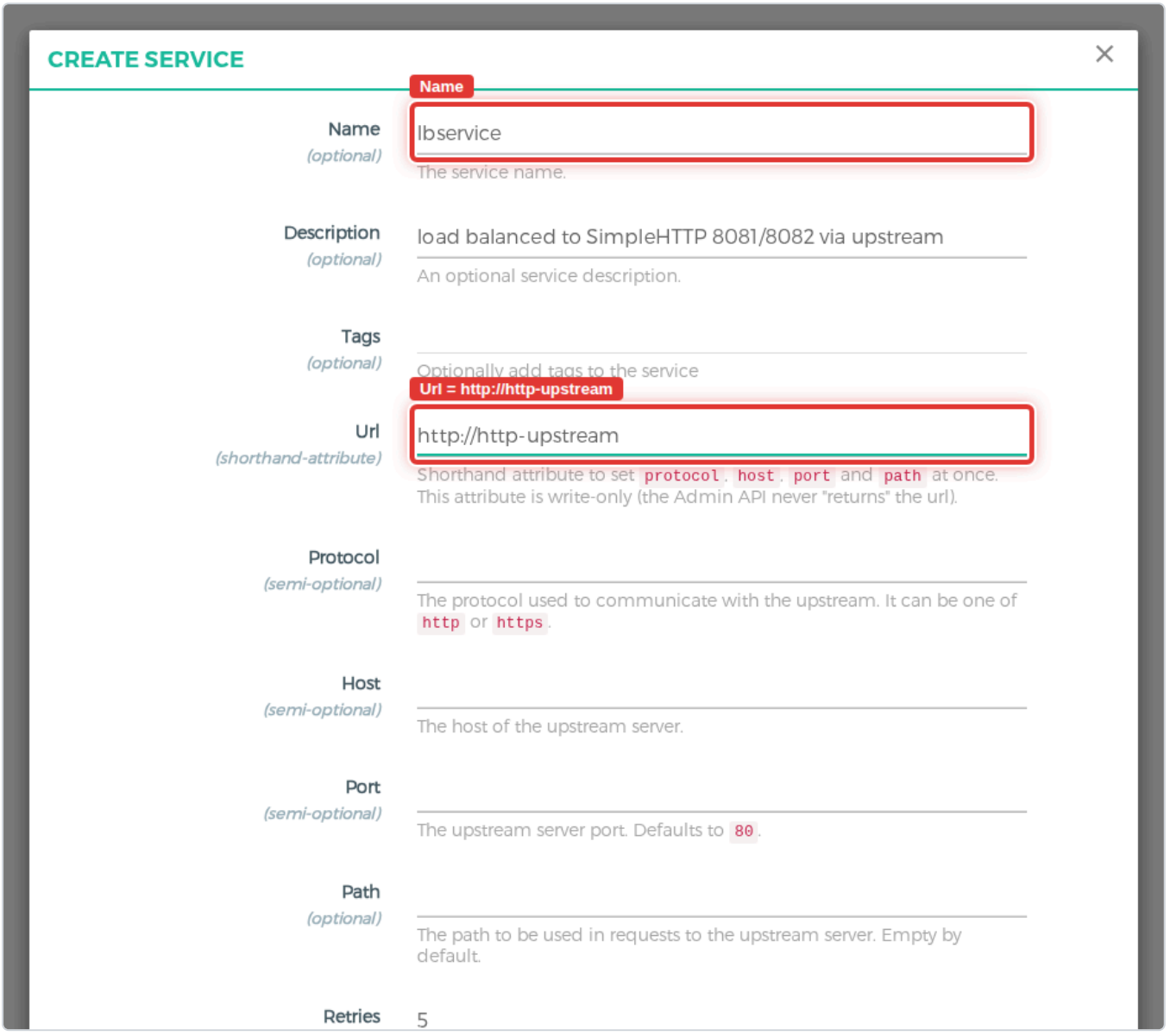
สร้าง upstream ชื่อ http-upstream (ส่วนล่าง)



เพิ่ม target 10.0.10.51:8081 (weight 100) และ 10.0.10.51:8082 (weight 100)



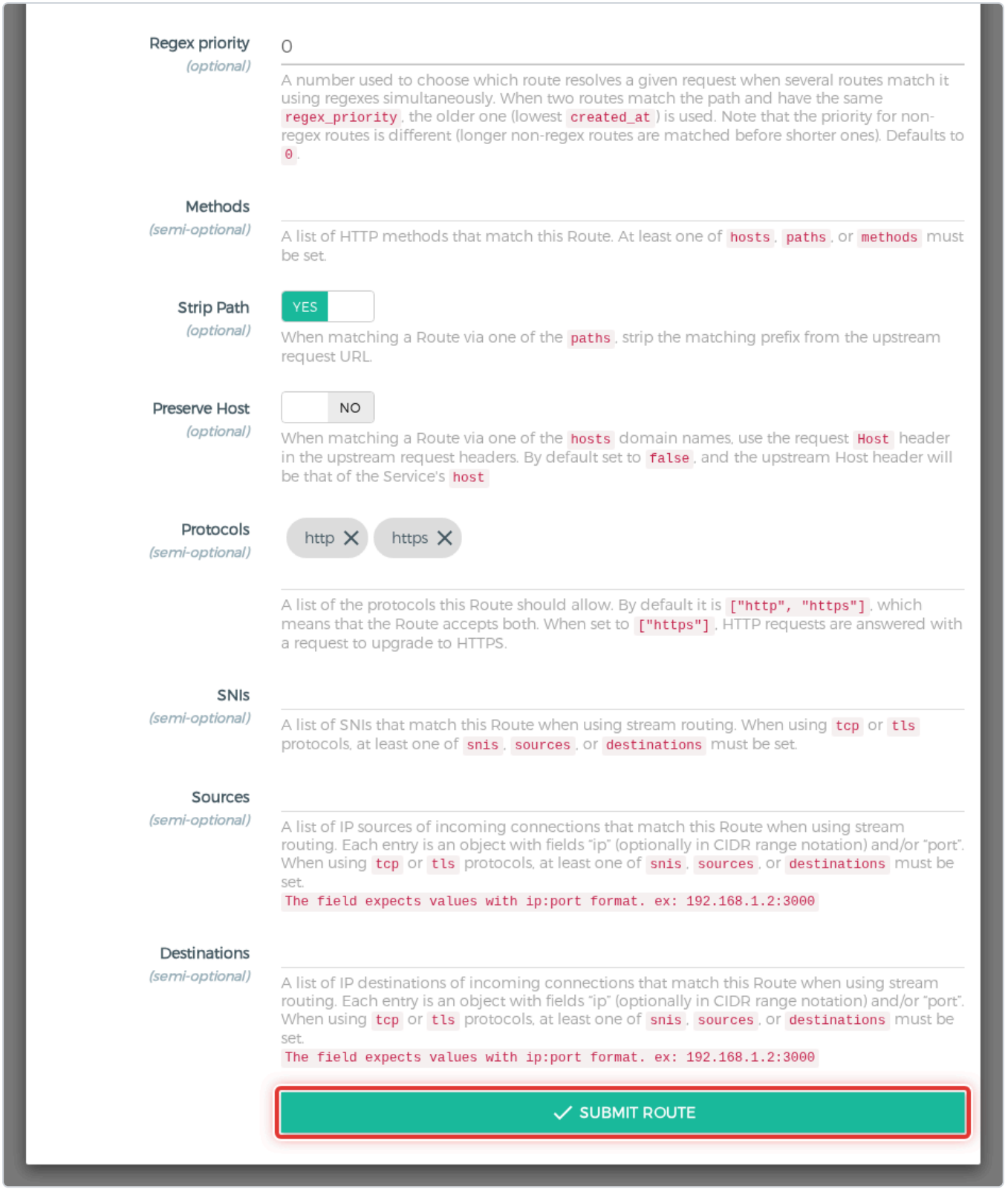
targets ทั้งสองของ http-upstream



สร้าง service lbservice ชี้ไปที่ http://http-upstream (ชื่อ upstream แทน host)



Route lb-route: paths = /lb (ส่วนบน)

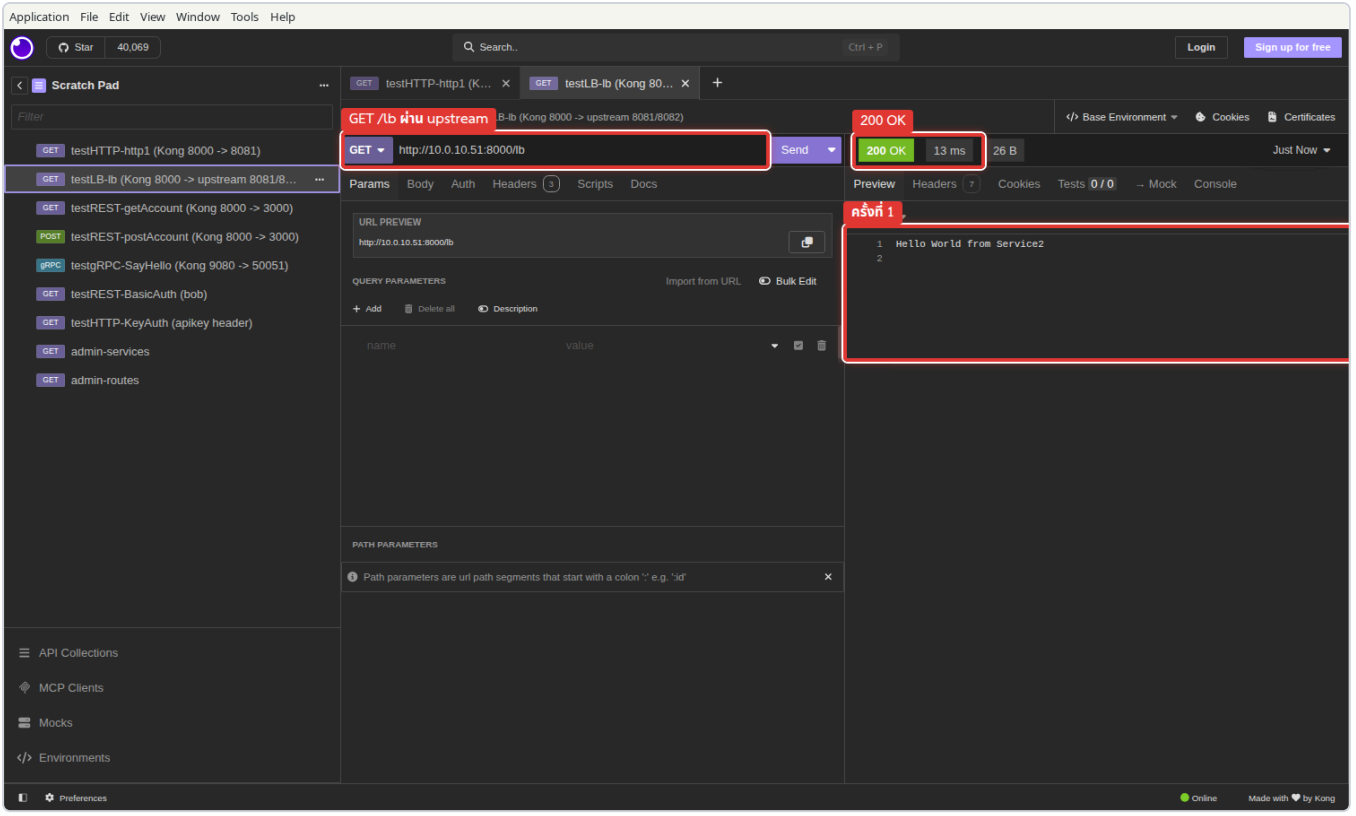


Route lb-route: paths = /lb (ส่วนล่าง)

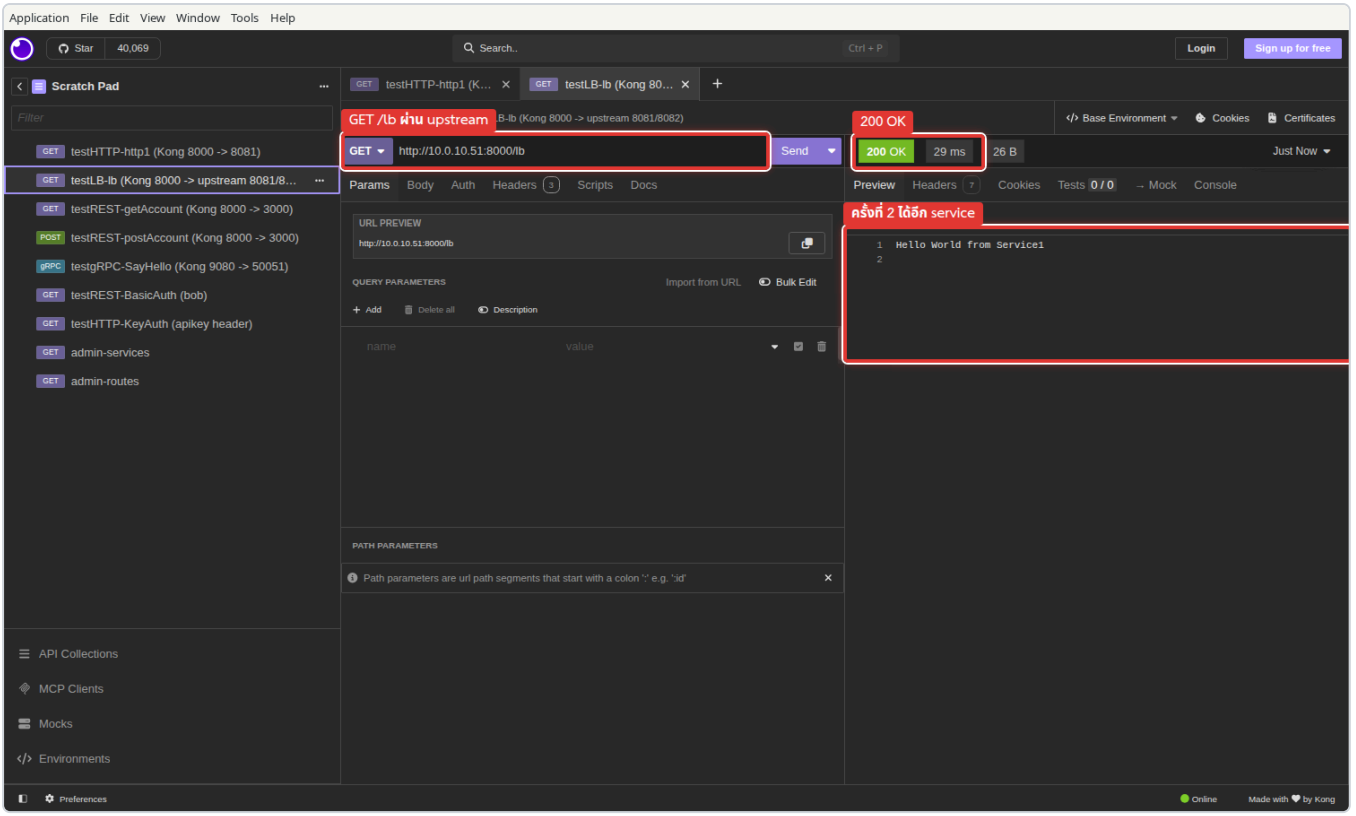
```
codespace@codespace_launcher: bash — 17-test-lb-via-kong

$ for i in 1 2 3 4 5 6; do curl -s http://10.0.10.51:8000/lb; done
Hello World from Service2
Hello World from Service1
Hello World from Service2
Hello World from Service1
Hello World from Service2
Hello World from Service1
[exit 0]
```

เรียก /lb 6 ครั้ง Kong สลับ (round-robin) ระหว่าง Service1 และ Service2

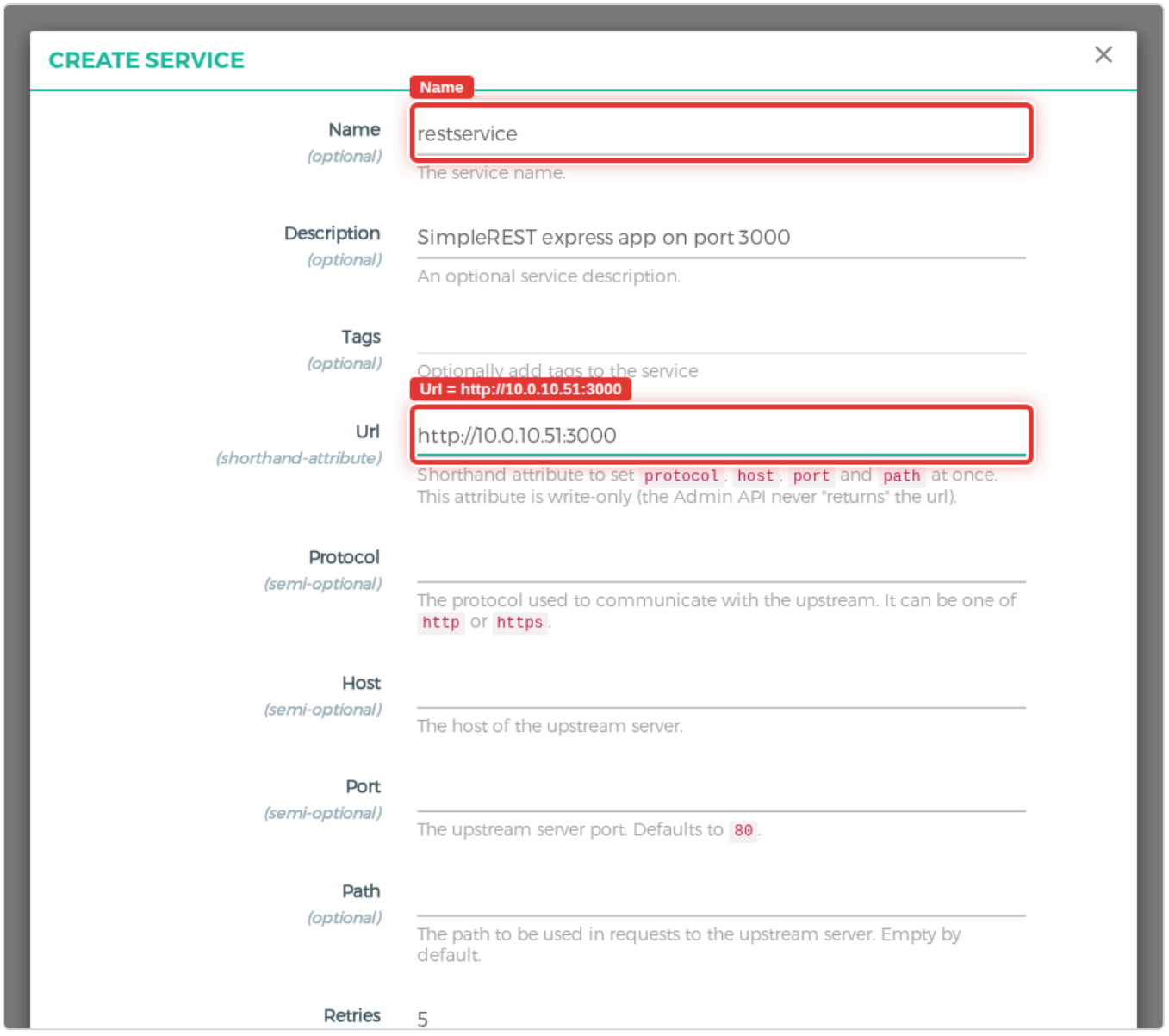


Insomnia: GET /lb ครั้งที่ 1

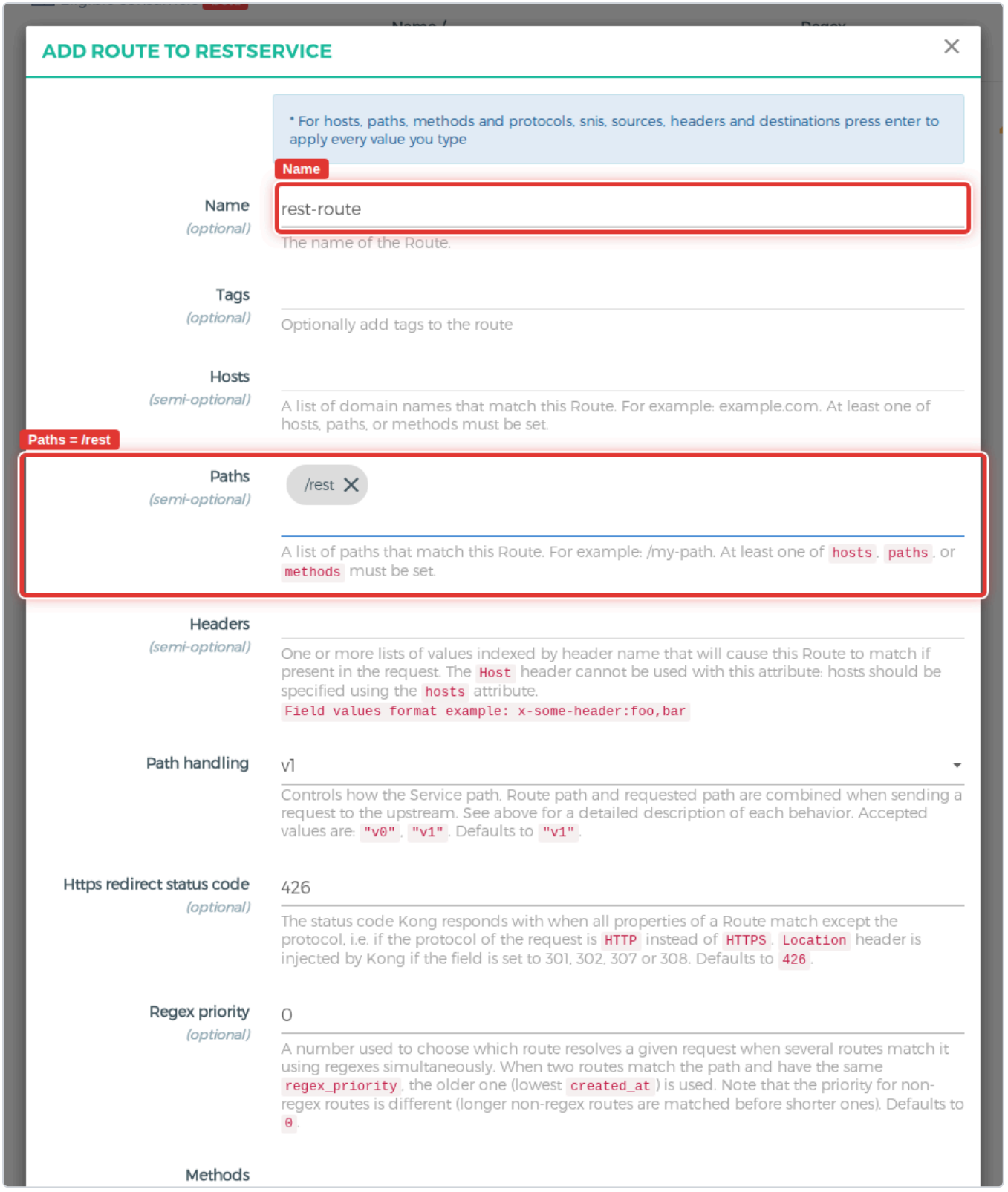


Insomnia: GET /lb **ครั้งที่** 2 **ได้อีก** service **หนึ่ง**

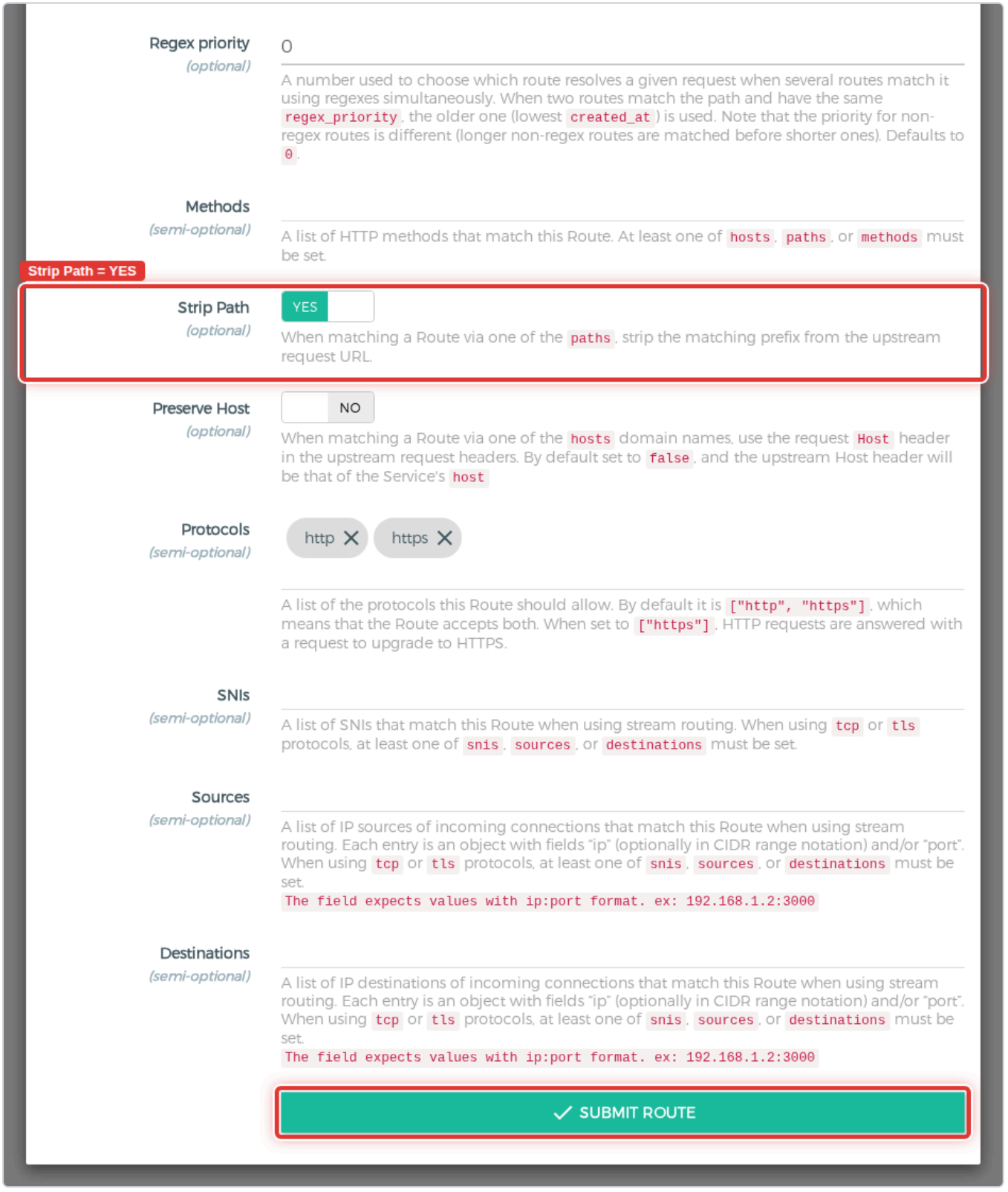
7. Step 5: Service และ Route สำหรับ REST (restservice → :3000)



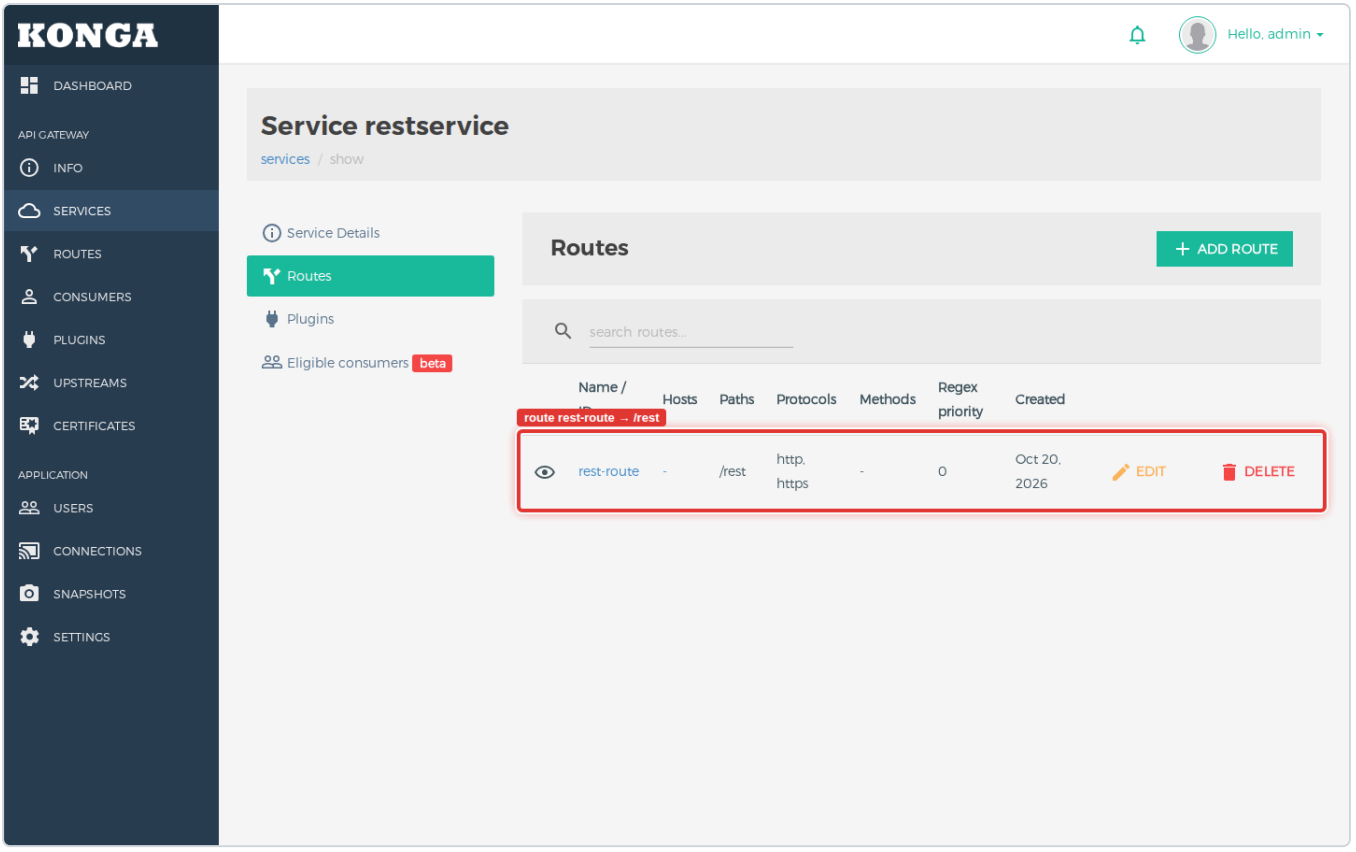
สร้าง service restservice url = http://10.0.10.51:3000



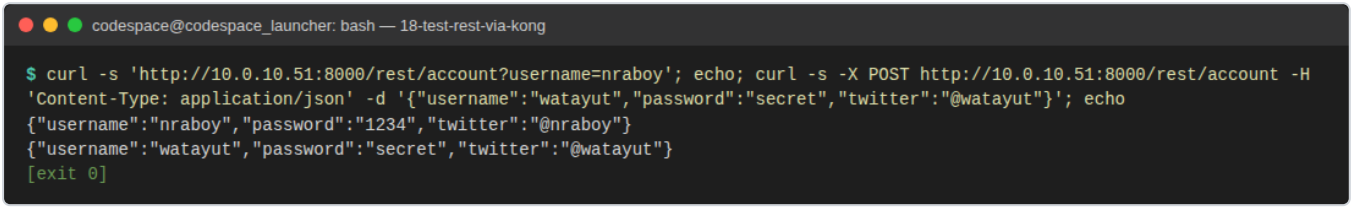
Route rest-route: paths = /rest, strip path = YES (Kong ตัด /rest ออกก่อนส่งต่อ จึงเรียก /rest/account → /account) (ส่วนบน)



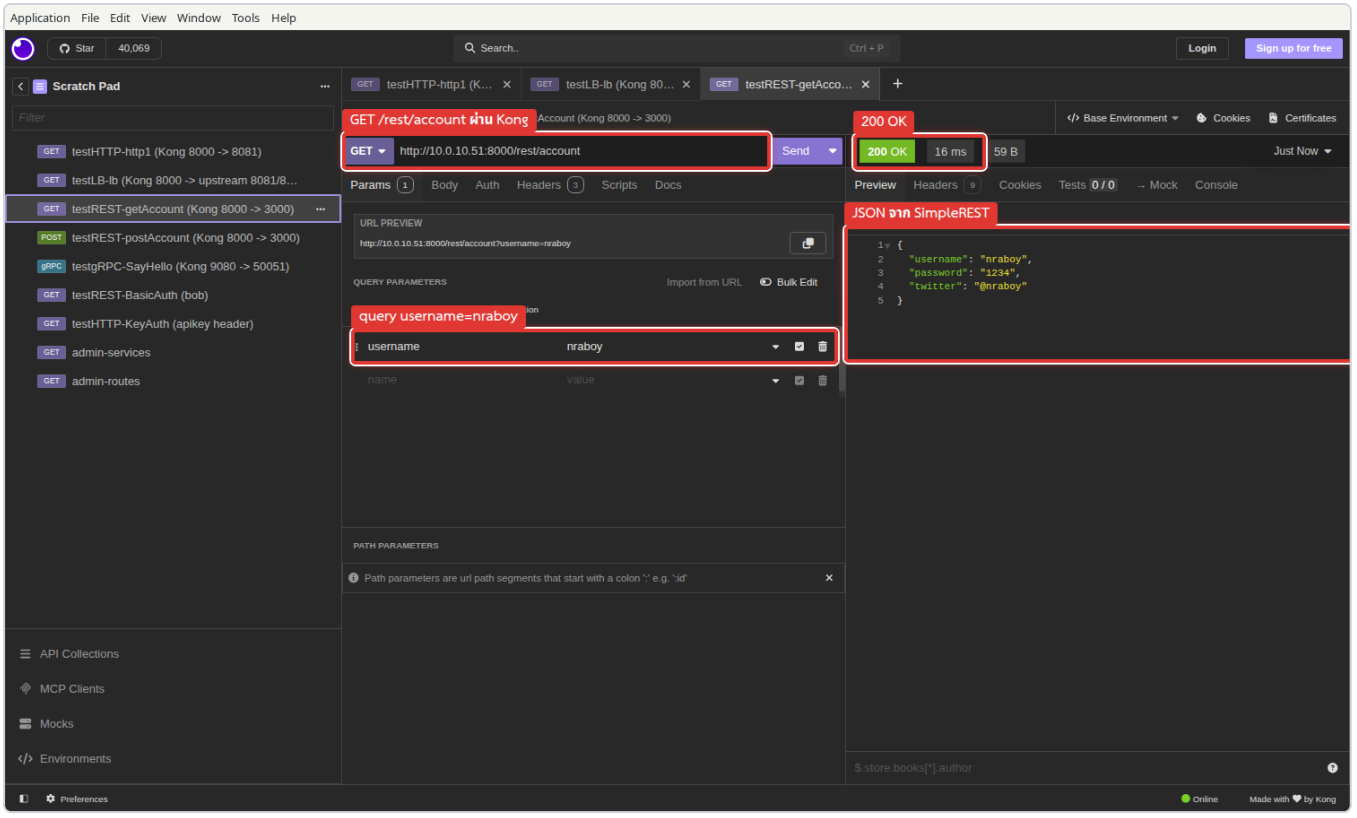
Route rest-route: paths = /rest, strip path = YES (Kong ตัด /rest ออกก่อนส่งต่อ จึงเรียก /rest/account → /account) (ส่วนล่าง)



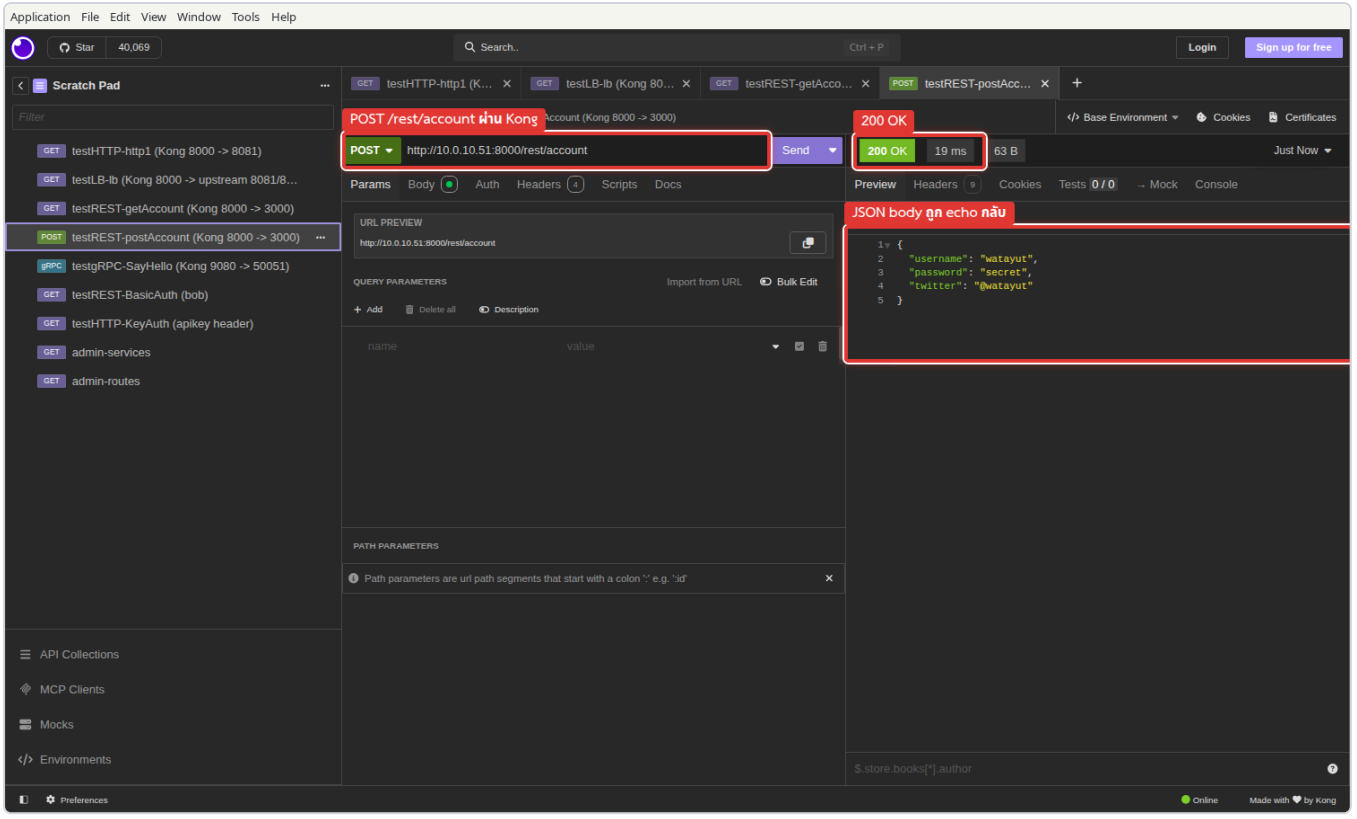
Route ของ restservice



GET /rest/account?username=nraboy และ POST /rest/account ผ่าน Kong



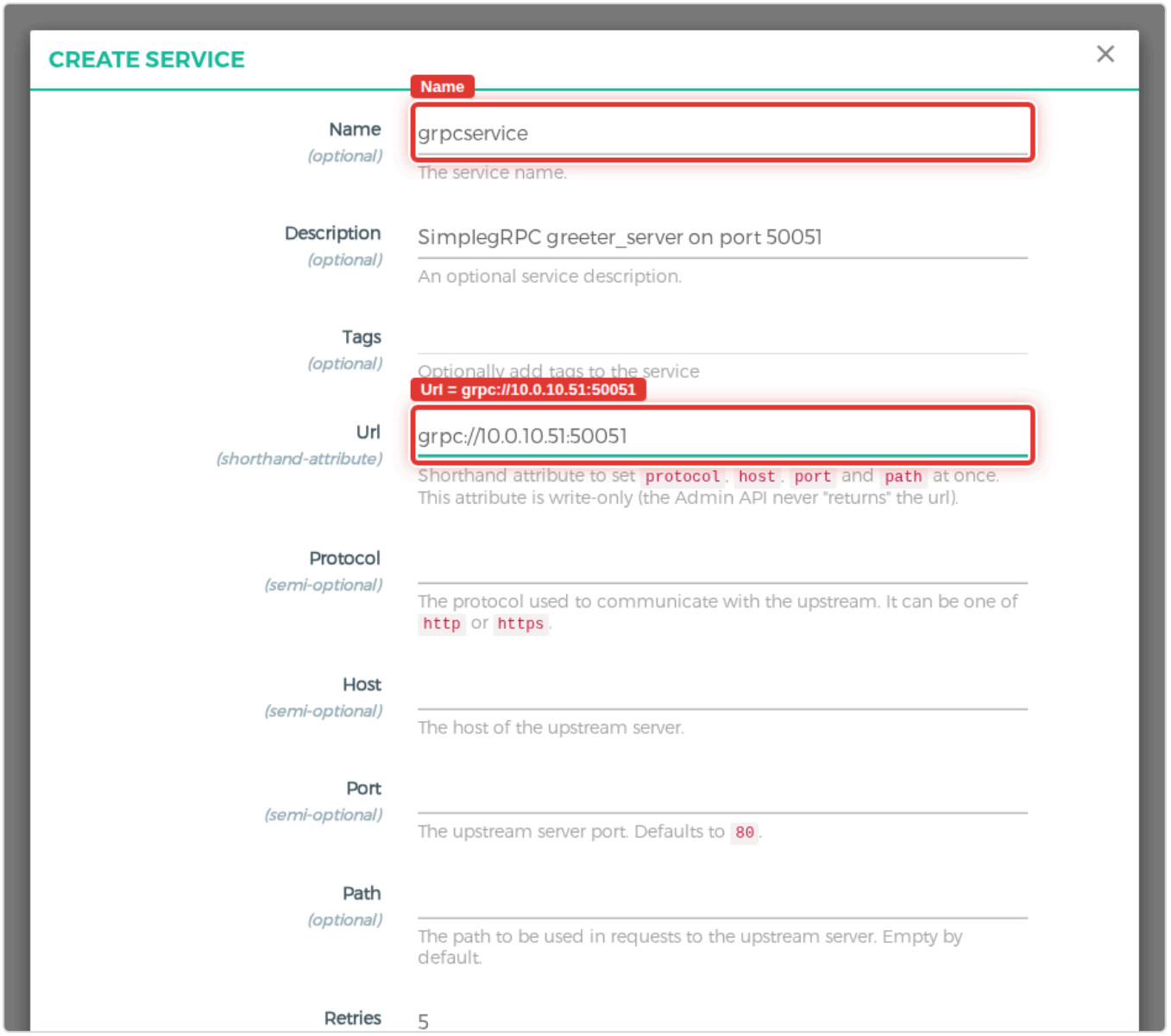
Insomnia: GET http://10.0.10.51:8000/rest/account?username=nraboy → 200 OK JSON



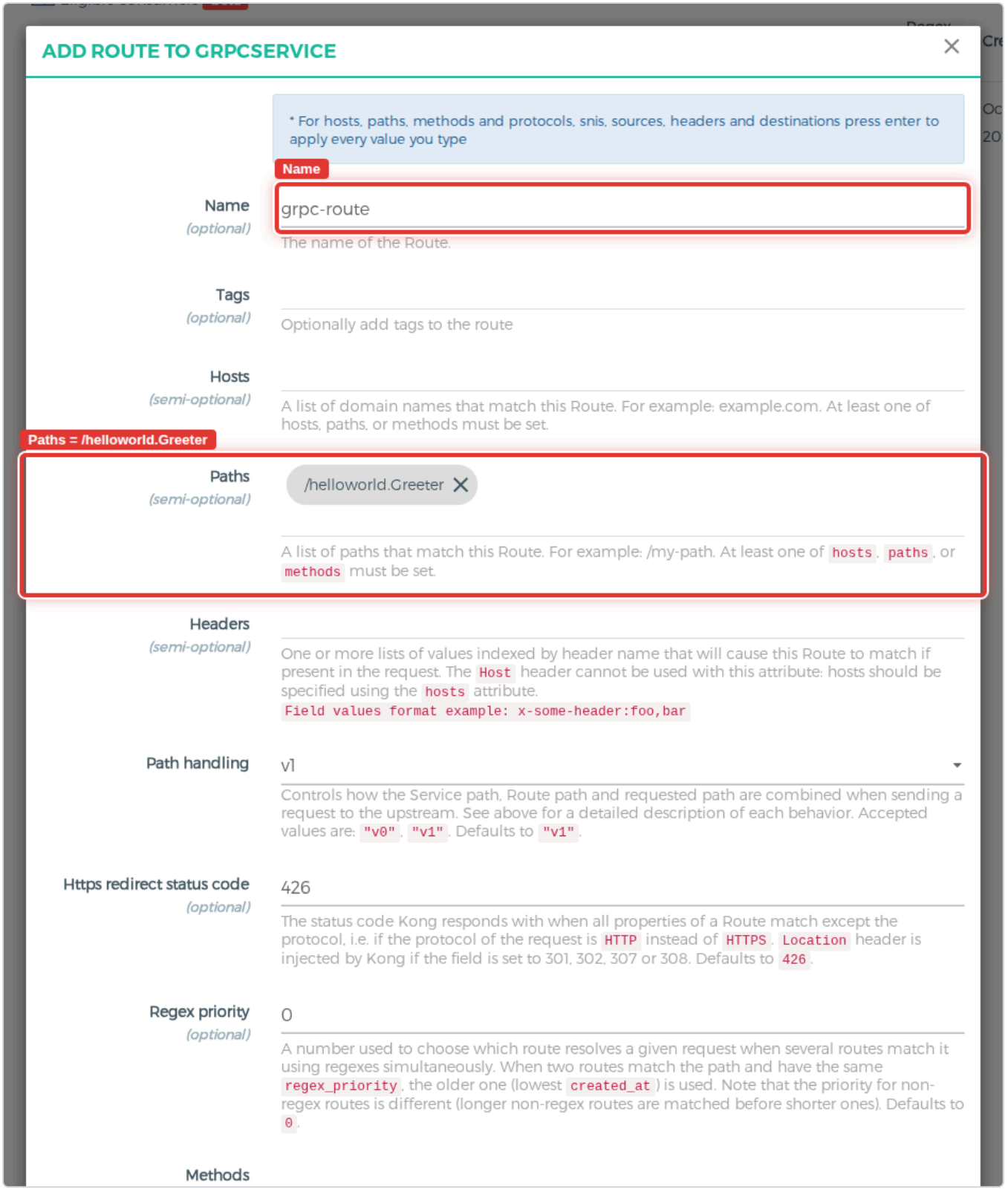
Insomnia: POST http://10.0.10.51:8000/rest/account (JSON body) → echo **กลับมา**

8. Step 6: Service และ Route สำหรับ gRPC (grpcservice → :50051)

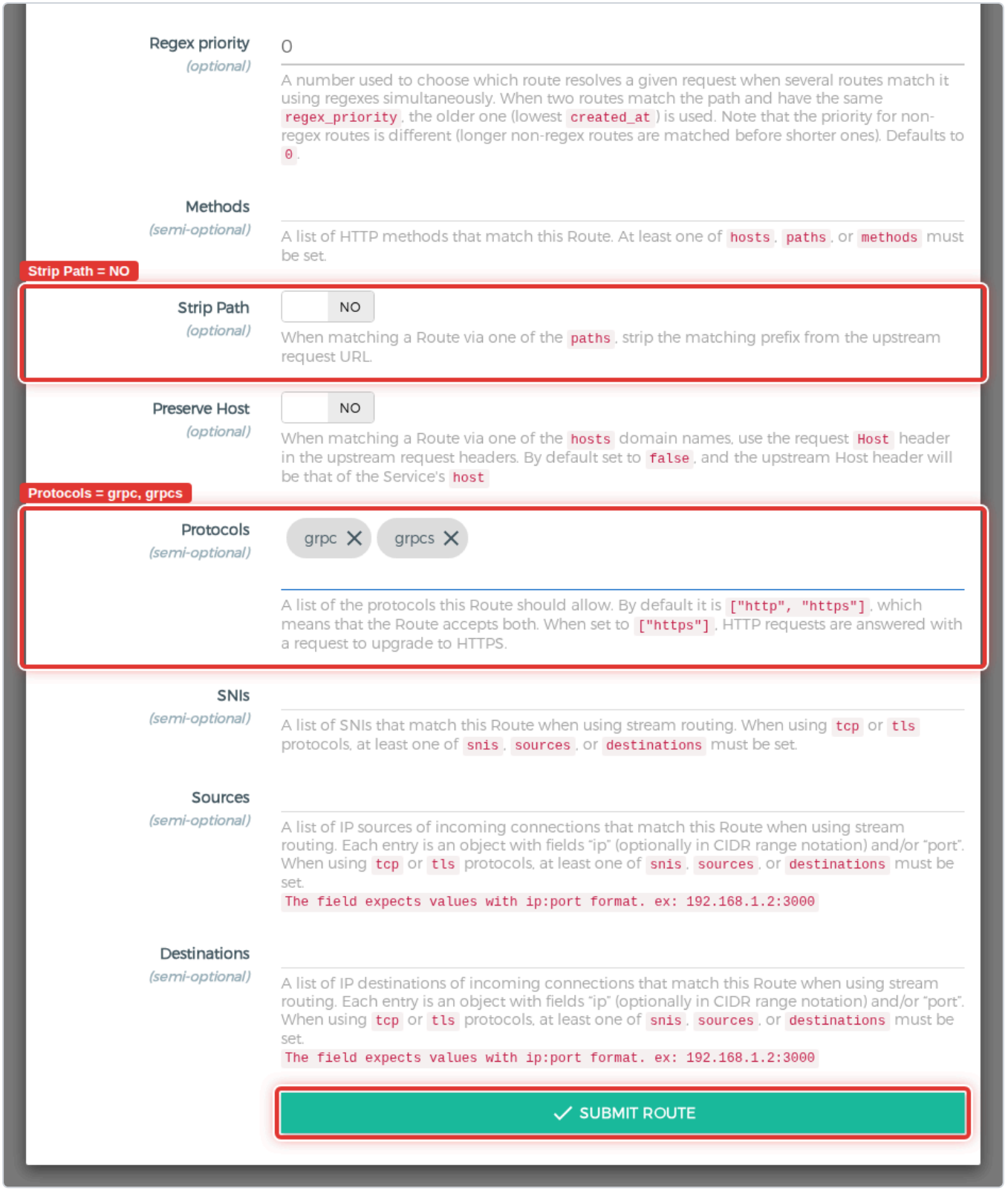
Kong รับ gRPC ที่ port 9080 (http2) ซึ่งคำสั่งใน PDF publish ไว้ที่ `127.0.0.1:9080` เท่านั้น การทดสอบ gRPC จึงเรียกที่ `localhost:9080` ส่วน service ปลายทางยังชี้ไปที่ IP ของเครื่อง สำหรับ route ต้องตั้ง protocols = grpc, grpcs และ strip path = NO เพราะ path `/helloworld.Greeter` คือชื่อ service ของ gRPC ที่ต้องส่งต่อไปทั้งก้อน



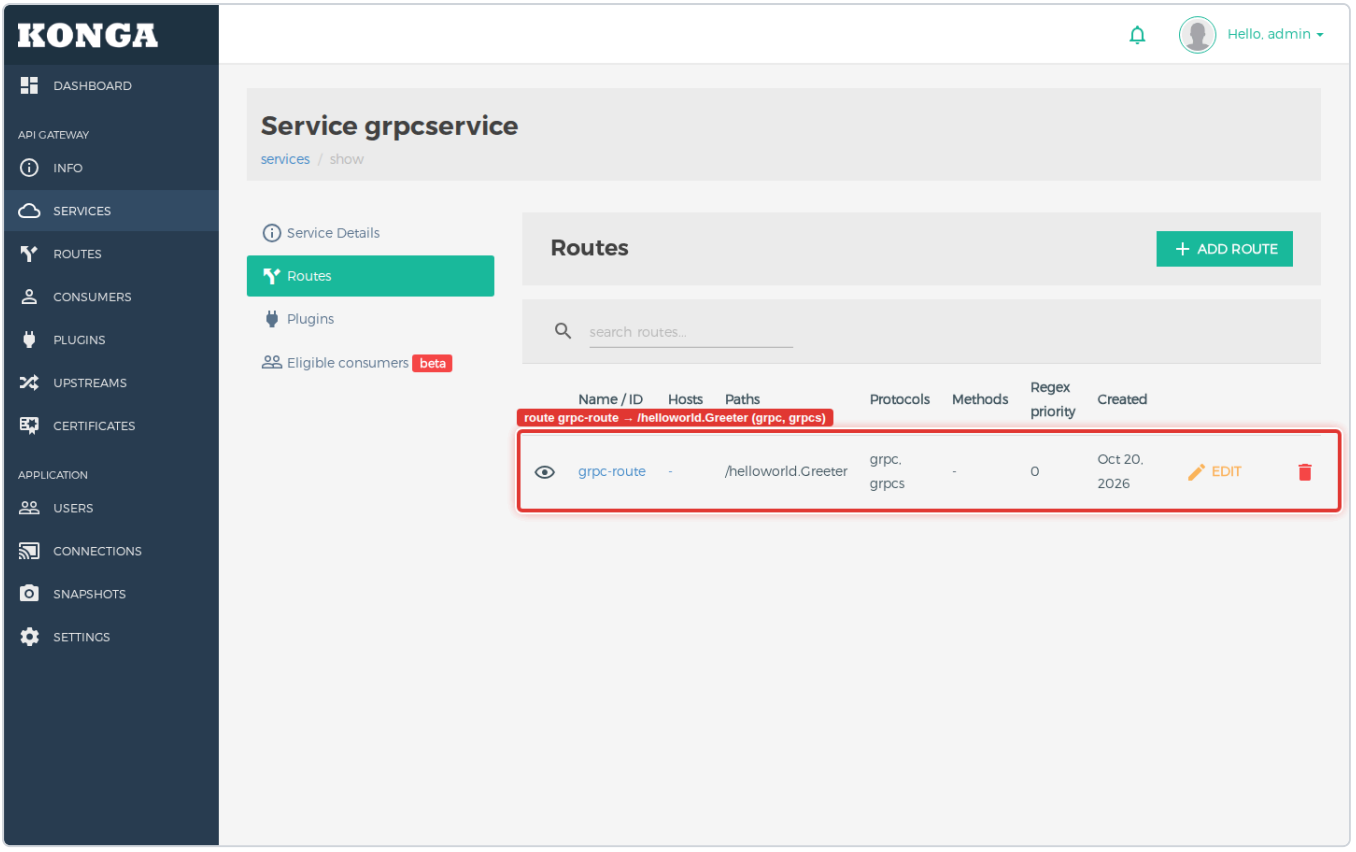
สร้าง service grpcservice url = grpc://10.0.10.51:50051



Route grpc-route: paths = /helloworld.Greeter, protocols = grpc, grpcs, strip path = NO (ส่วนบน)



Route grpc-route: paths = /helloworld.Greeter, protocols = grpc, grpcs, strip path = NO (ส่วนล่าง)

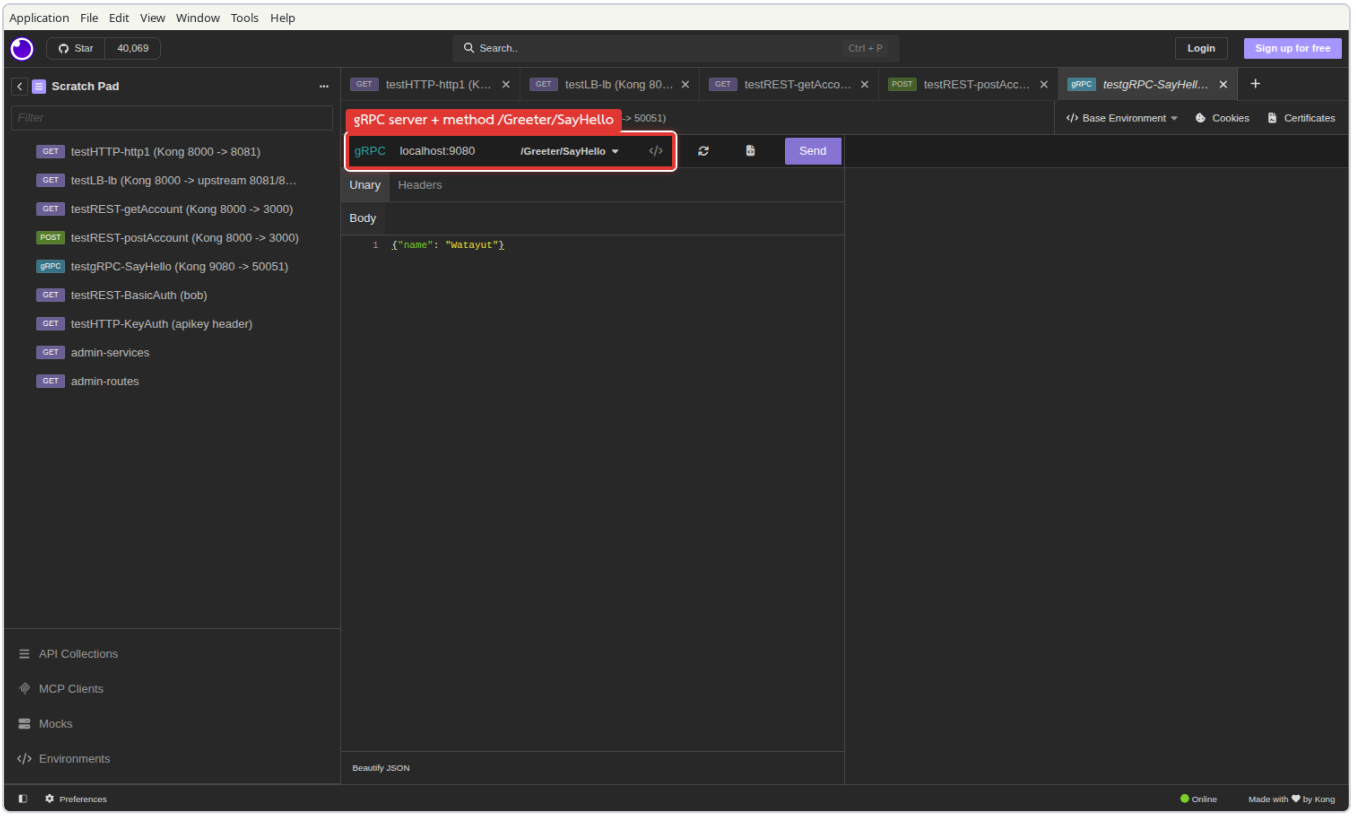


Route ของ grpcservice

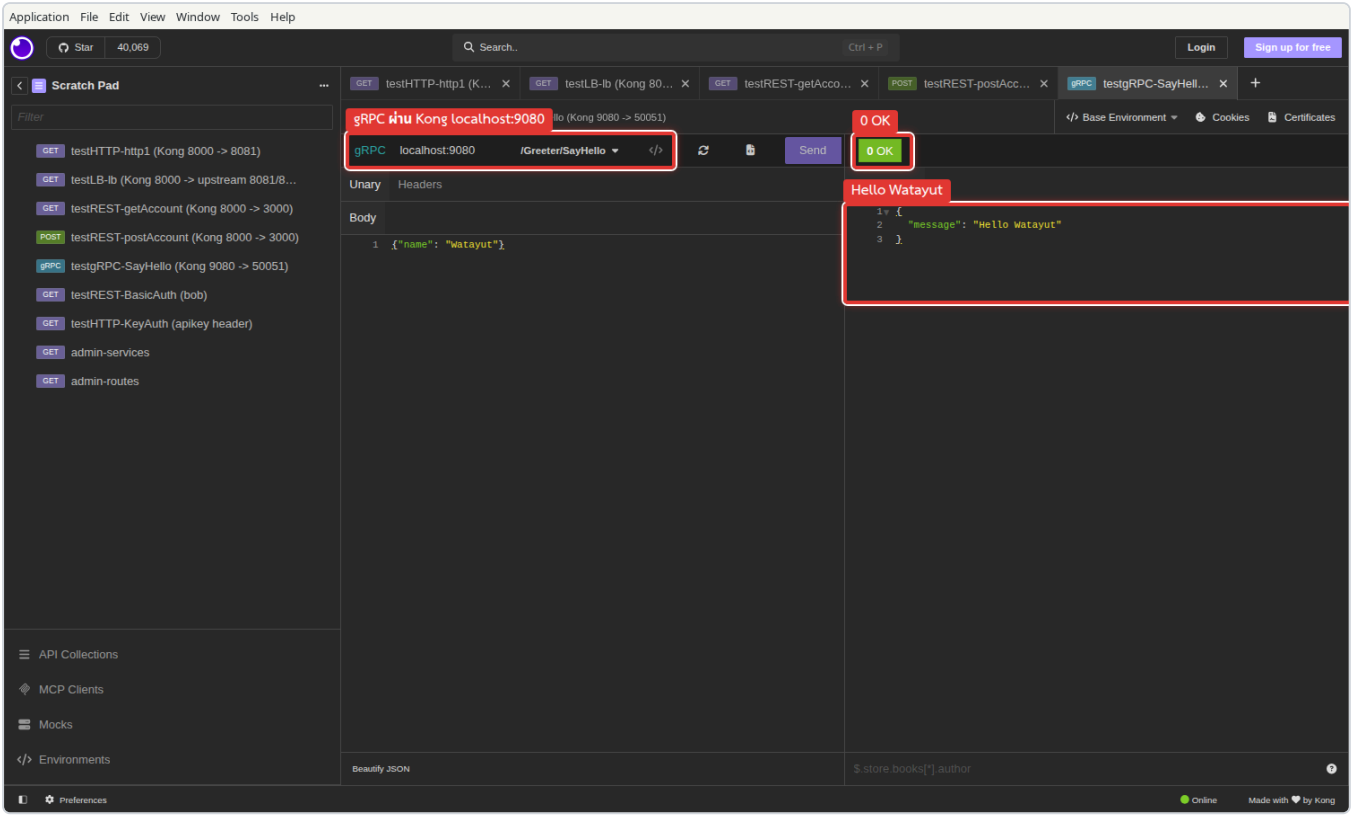
```
codespace@codespace_launcher: bash — 19-test-grpc-via-kong

$ cd SimplegRPC/node && grpcurl -plaintext -import-path . -proto helloworld.proto -d '{"name":"Watayut"}' localhost:9080 hello
world.Greeter/SayHello
{
  "message": "Hello Watayut"
}
[exit 0]
```

grpcurl เรียก helloworld.Greeter/SayHello ผ่าน Kong :9080 → "Hello Watayut"

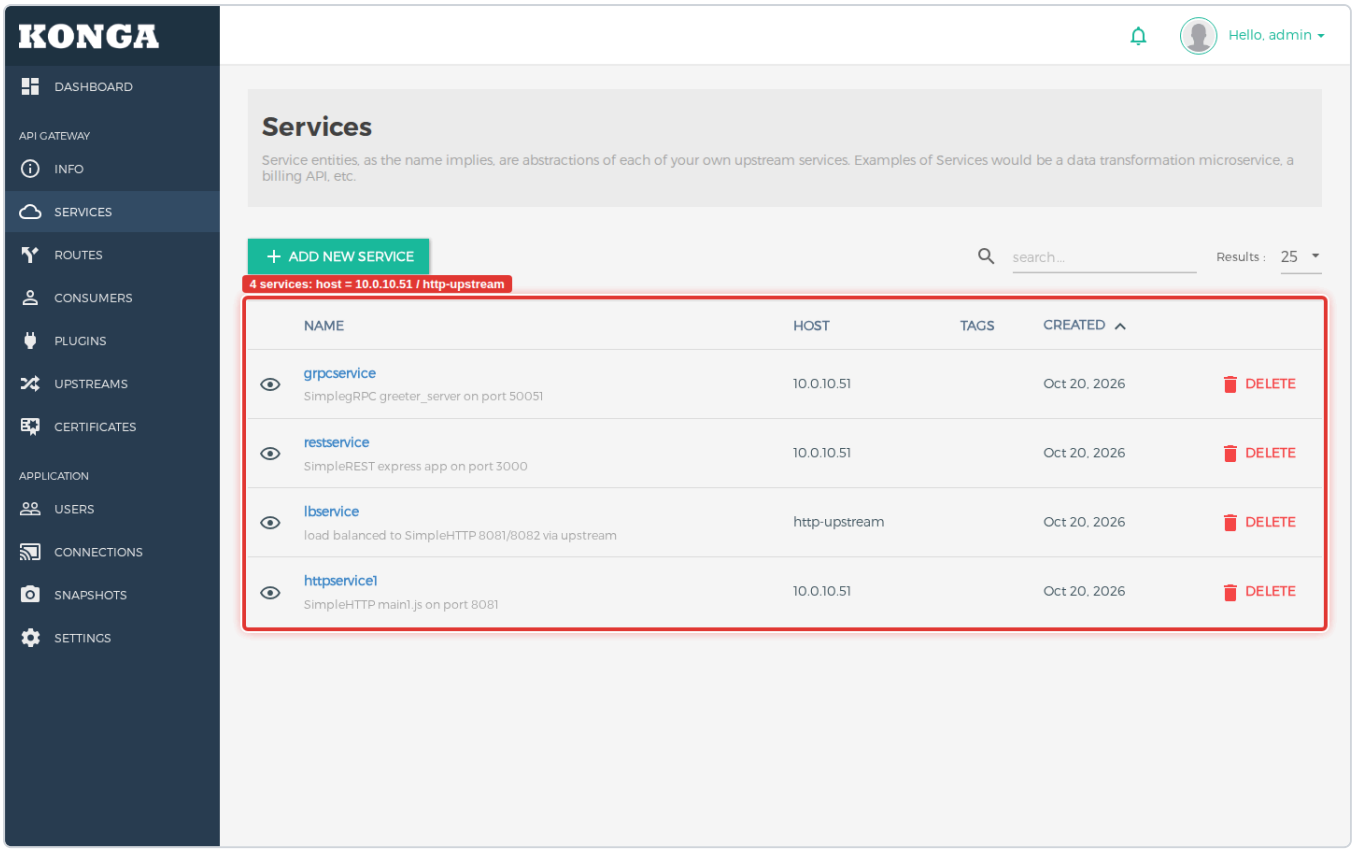


Insomnia gRPC request: proto helloworld.proto, method /Greeter/SayHello, server localhost:9080

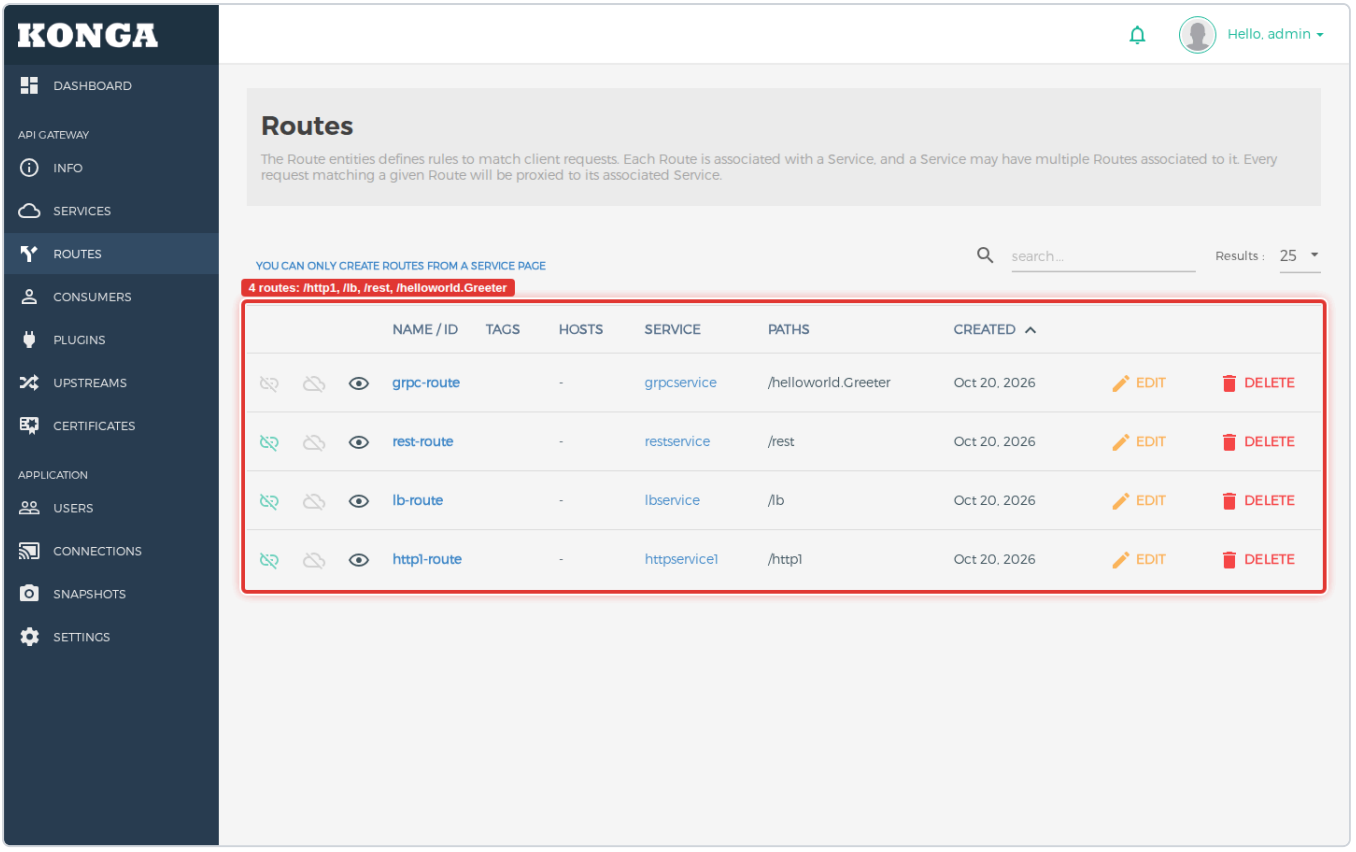


Insomnia: ส่ง {"name":"Watayut"} ผ่าน Kong ได้ response {"message":"Hello Watayut"} (0 OK)

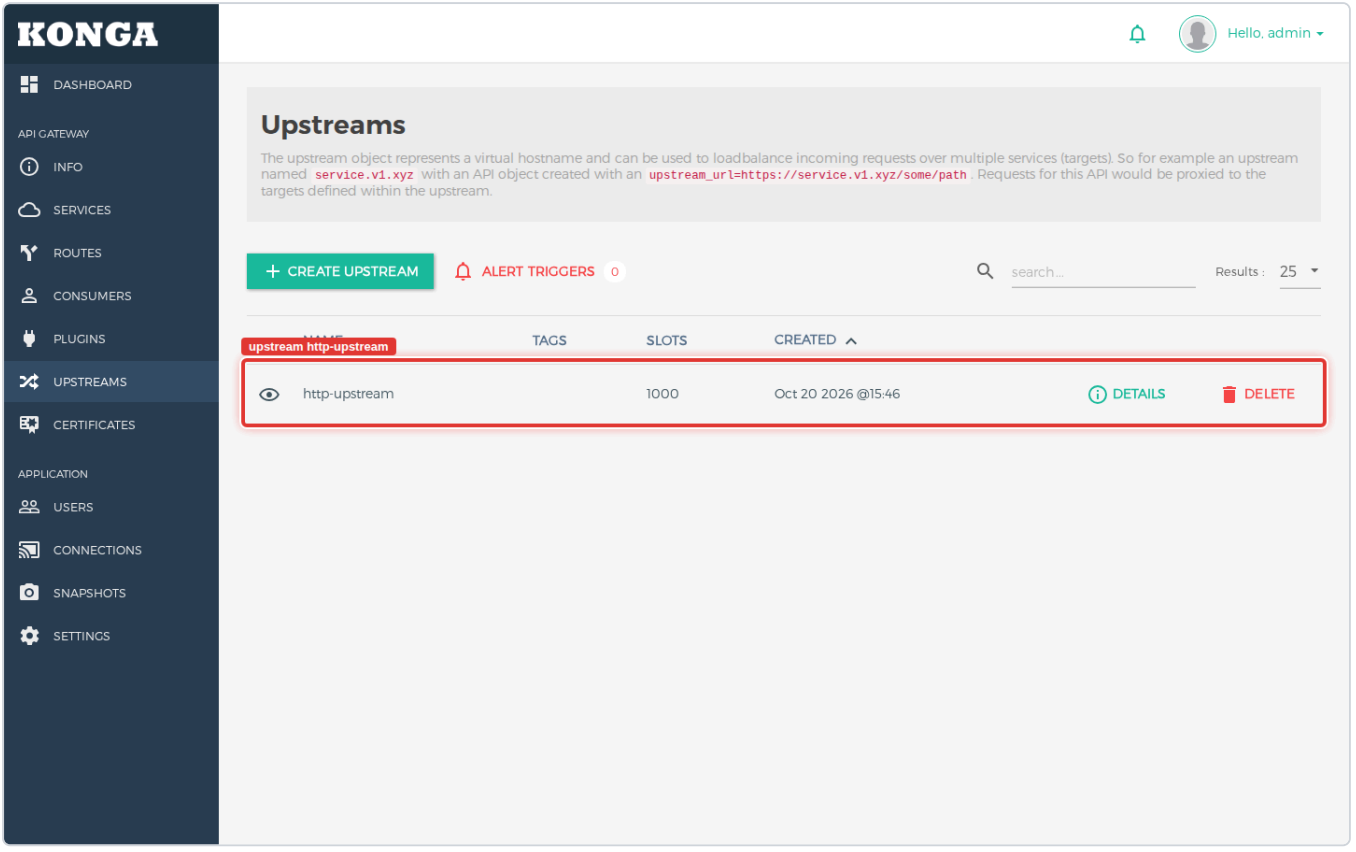
9. สรุป Services และ Routes ทั้งหมดใน Kong (หน้าจอที่โจทย์ต้องการ)



Konga → SERVICES: httpservice1, lbservice, restservice, grpcservice (host = 10.0.10.51 / http-upstream)



Konga → ROUTES: http1-route (/http1), lb-route (/lb), rest-route (/rest), grpc-route (/helloworld.Greeter)



Konga → UPSTREAMS: http-upstream

```
codespace@codespace_launcher: bash — 13-kong-services

$ curl -s http://localhost:8001/services | python3 -c "import sys,json; [print(f\"{s['name']:14} {s['protocol']}://{s['hos
t']}:{s['port']}\") for s in json.load(sys.stdin)['data']]"
httpservice1   http://10.0.10.51:8081
restservice    http://10.0.10.51:3000
lbservice      http://http-upstream:80
grpcservice    grpc://10.0.10.51:50051
[exit 0]
```

Kong Admin API: GET /services

```
codespace@codespace_launcher: bash — 14-kong-routes

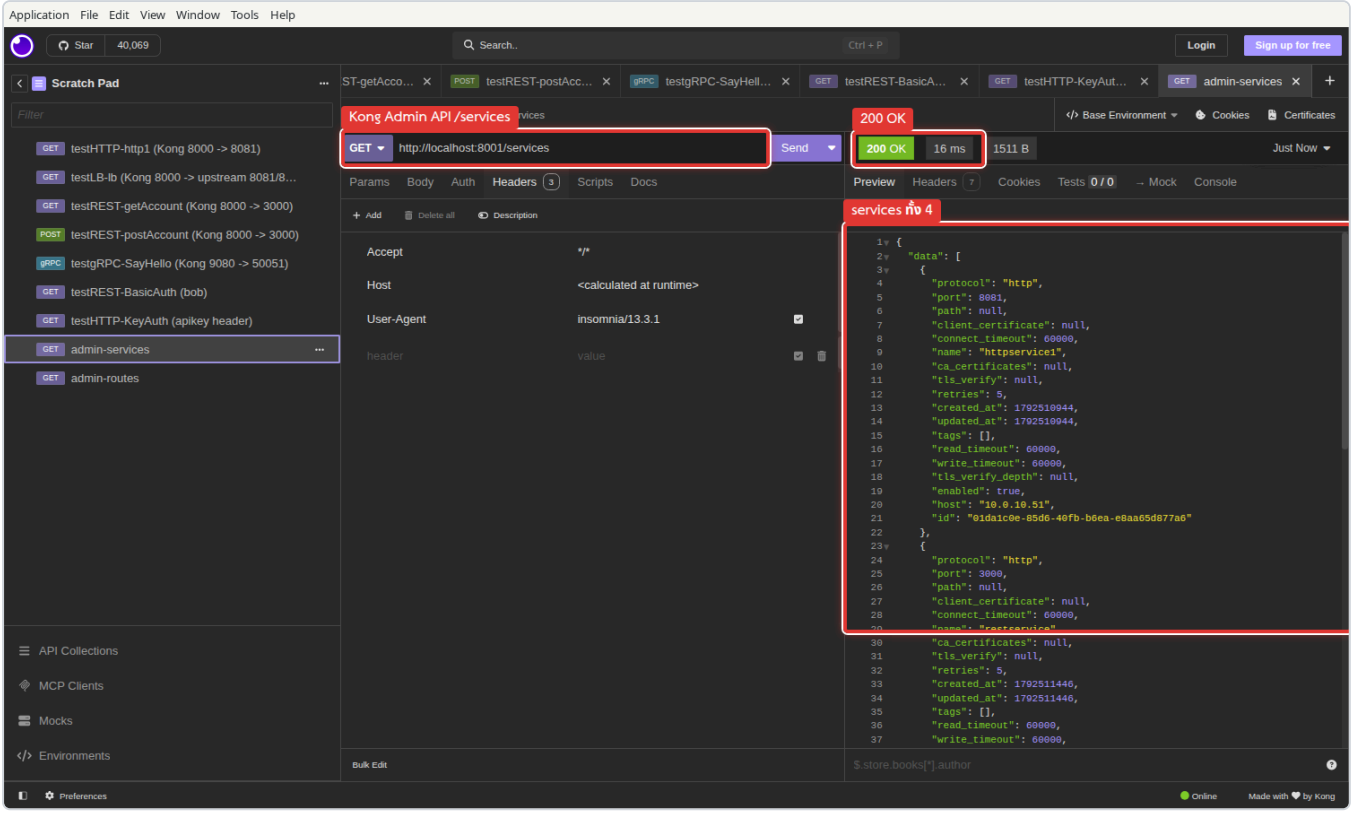
$ curl -s http://localhost:8001/routes | python3 -c "import sys,json; [print(f\"{r['name']:12} paths={r['paths']} protocols={r
['protocols']} strip_path={r['strip_path']}\") for r in json.load(sys.stdin)['data']]"
rest-route   paths=['/rest'] protocols=['http', 'https'] strip_path=True
lb-route     paths=['/lb'] protocols=['http', 'https'] strip_path=True
http1-route  paths=['/http1'] protocols=['http', 'https'] strip_path=True
grpc-route   paths=['/helloworld.Greeter'] protocols=['grpc', 'grpcs'] strip_path=False
[exit 0]
```

Kong Admin API: GET /routes

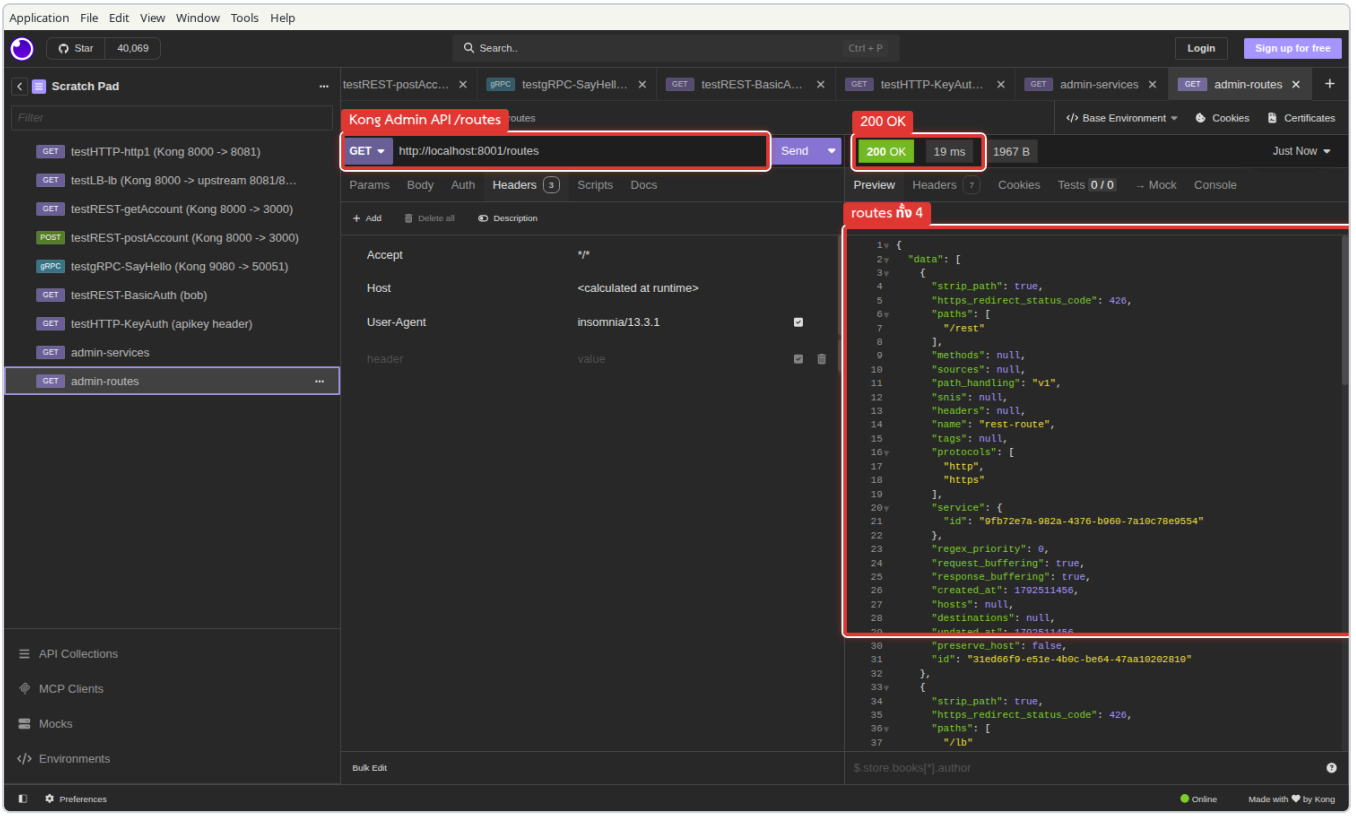
```
codespace@codespace_launcher: bash — 15-kong-upstream-targets

$ curl -s http://localhost:8001/upstreams/http-upstream/targets | python3 -c "import sys,json; [print(t['target'], 'weight', t
['weight']) for t in json.load(sys.stdin)['data']]"
10.0.10.51:8082 weight 100
10.0.10.51:8081 weight 100
[exit 0]
```

Kong Admin API: targets ของ http-upstream



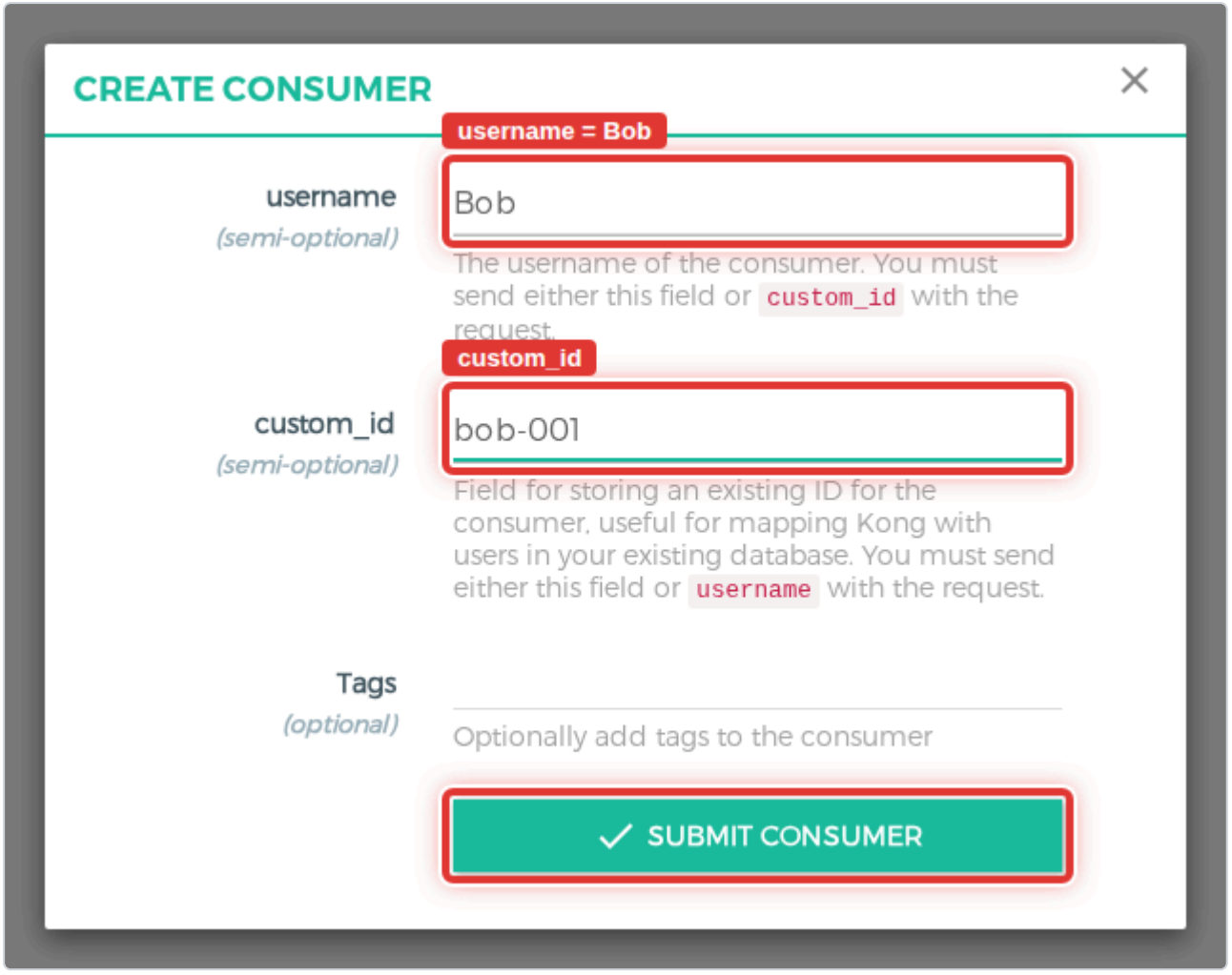
Insomnia: GET http://localhost:8001/services



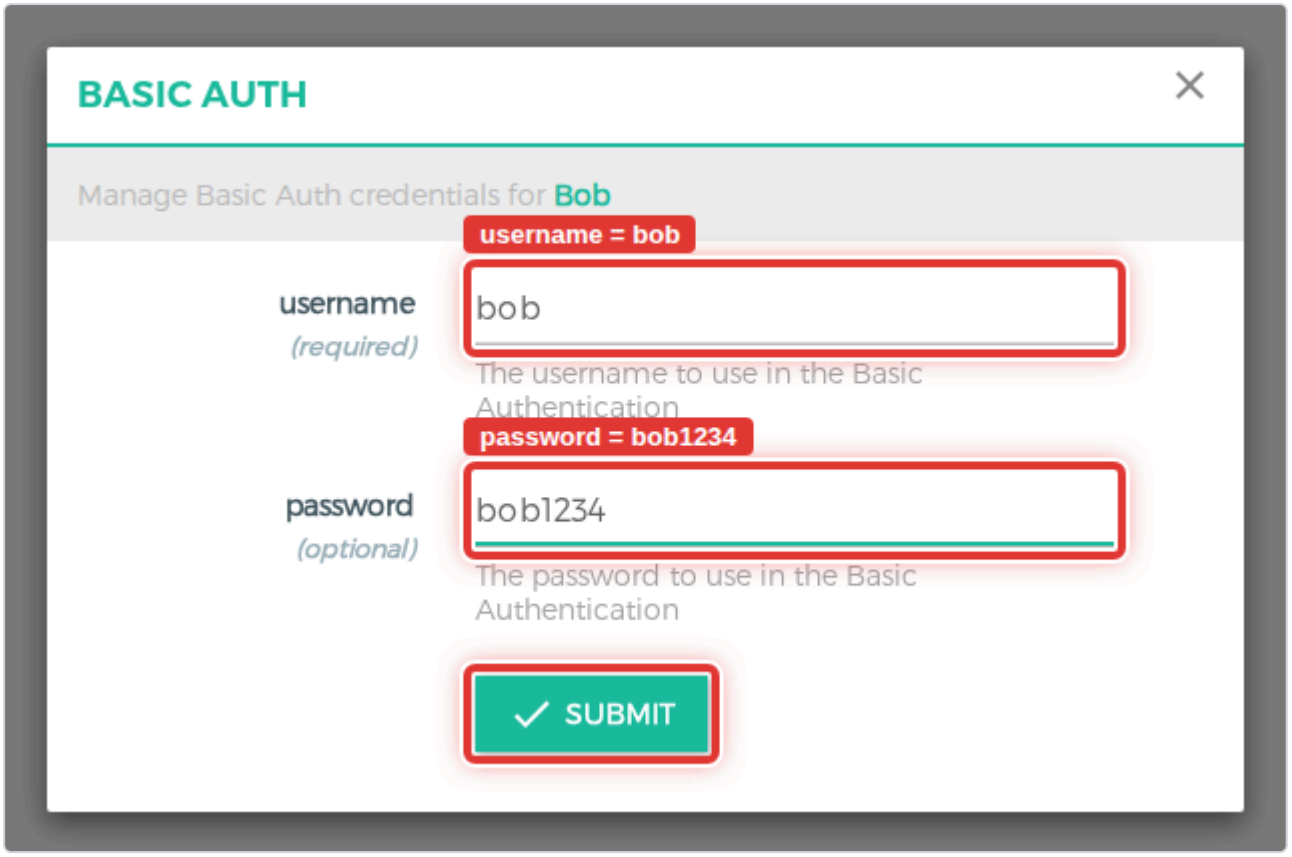
Insomnia: GET http://localhost:8001/routes

10. Step 7-8: Basic Authentication และ Key Authentication ของ consumer

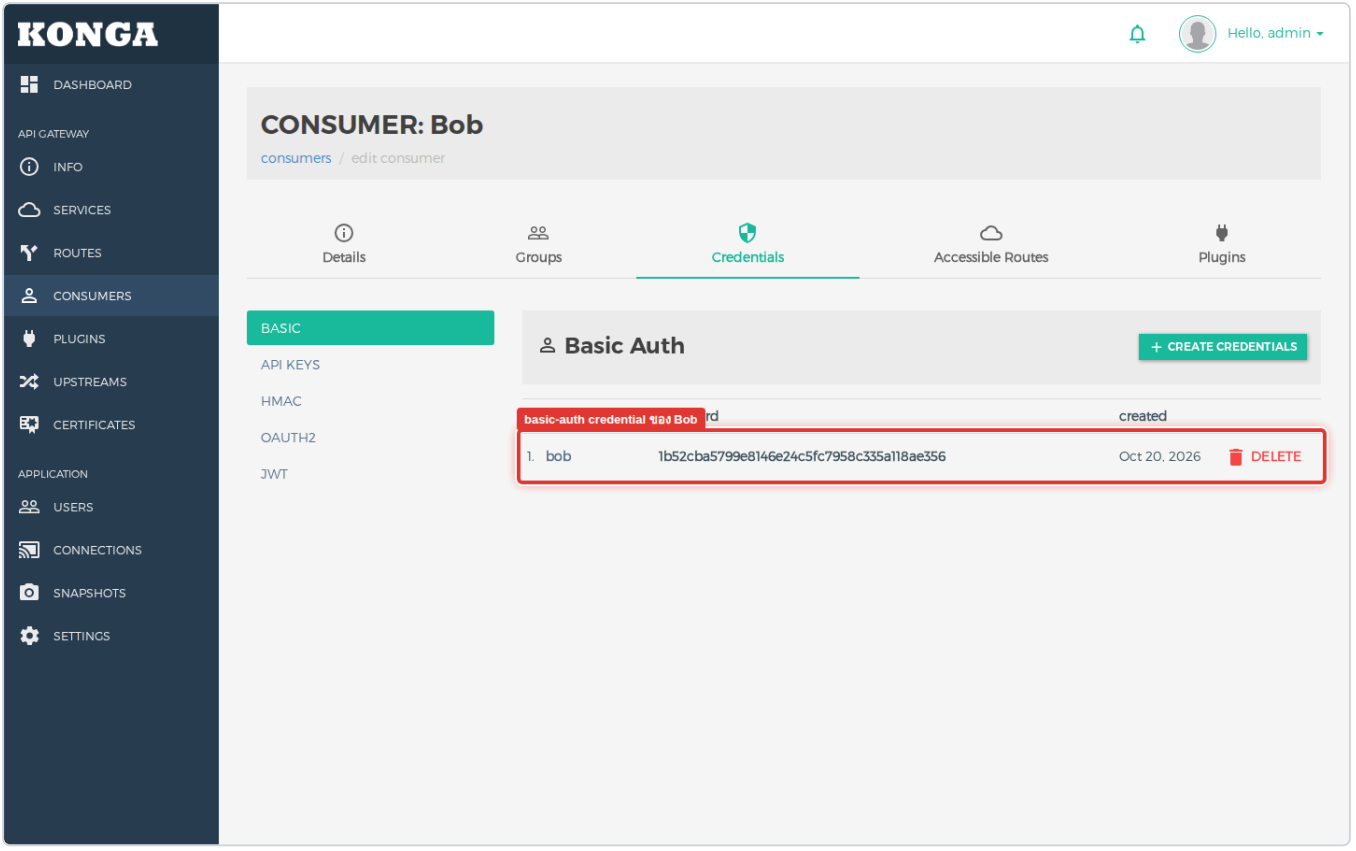
สร้าง consumer ชื่อ Bob ให้ credentials สองแบบ (basic-auth: bob / bob1234 และ api key: bob-secret-key-123) แล้วเปิด plugin basic-auth ที่ restservice และ key-auth ที่ httpservice1 จากนั้นทดสอบว่าถ้าไม่ส่ง credentials จะได้ 401 และเมื่อส่งถูกต้องจะได้ 200



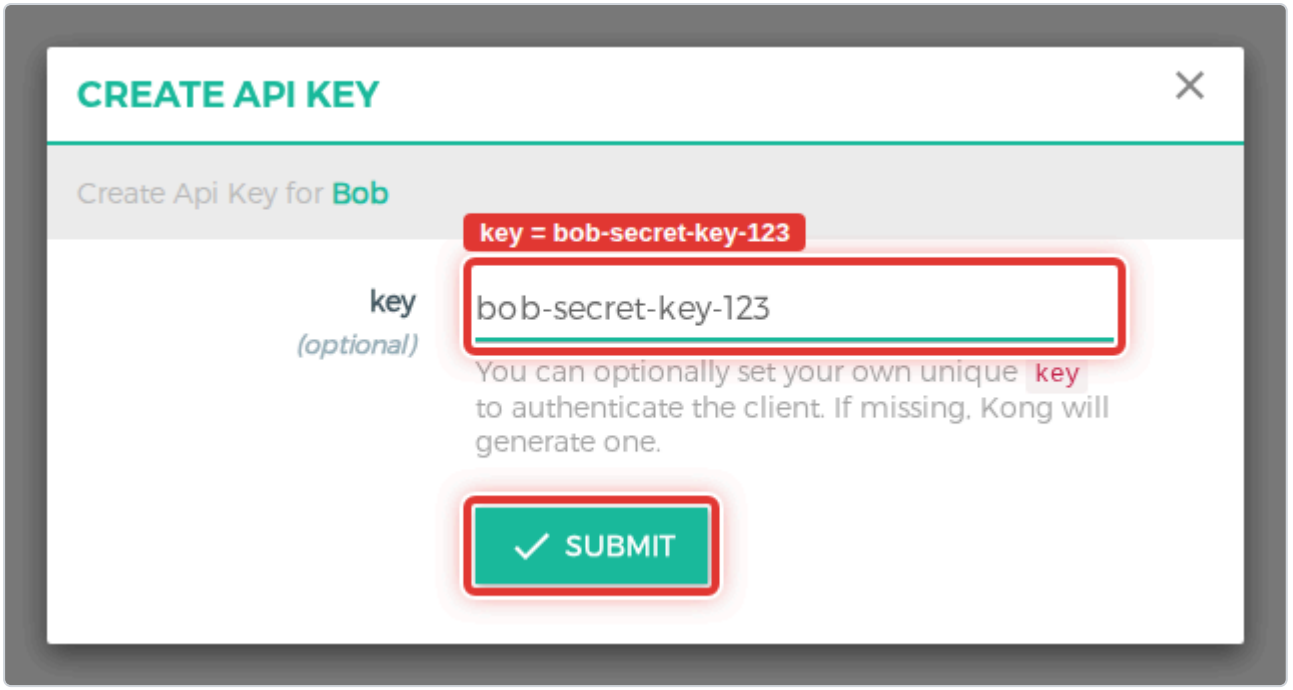
3.1 สร้าง consumer Bob (custom_id bob-001)



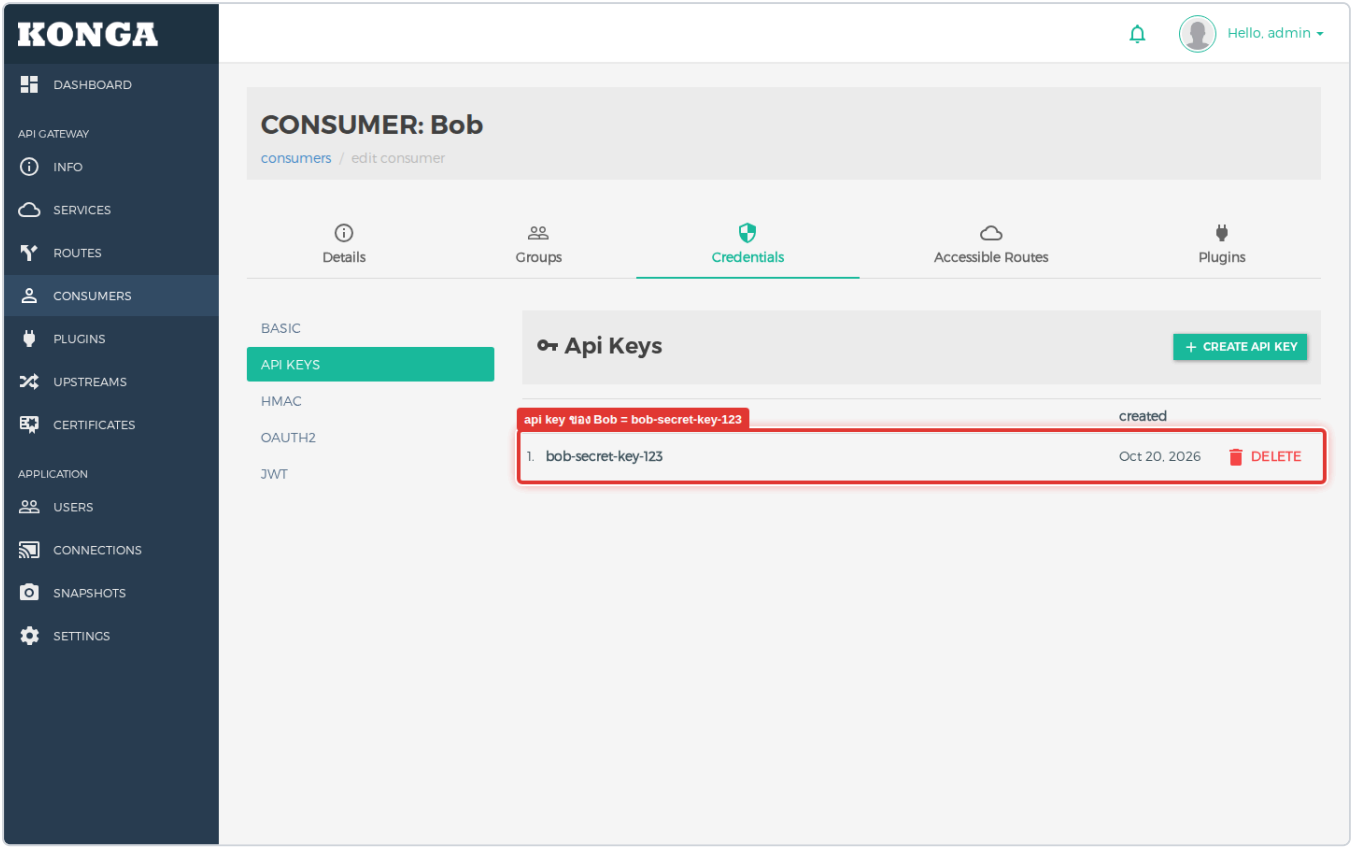
Credentials → BASIC: username bob, password bob1234



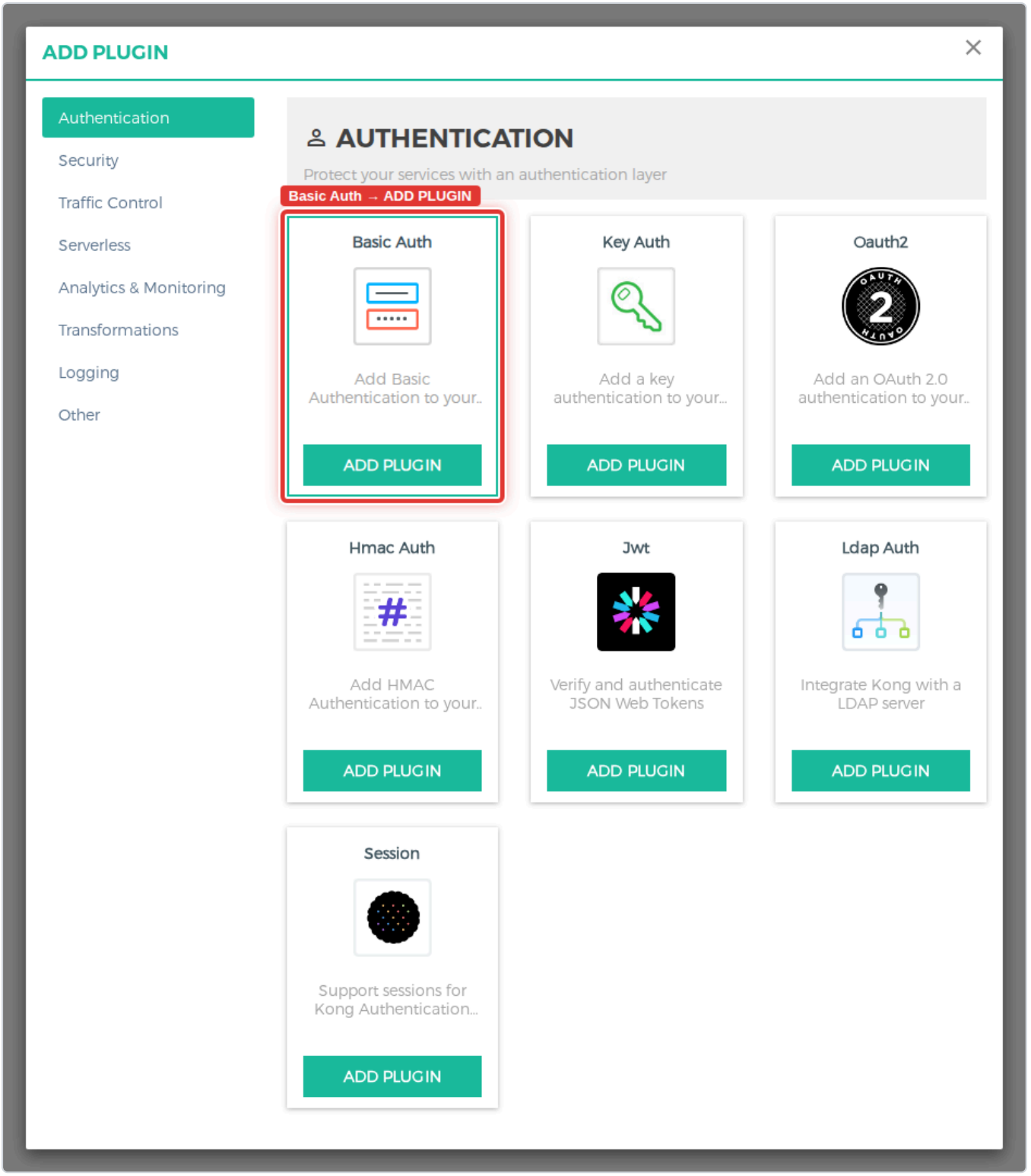
basic-auth credential ของ Bob



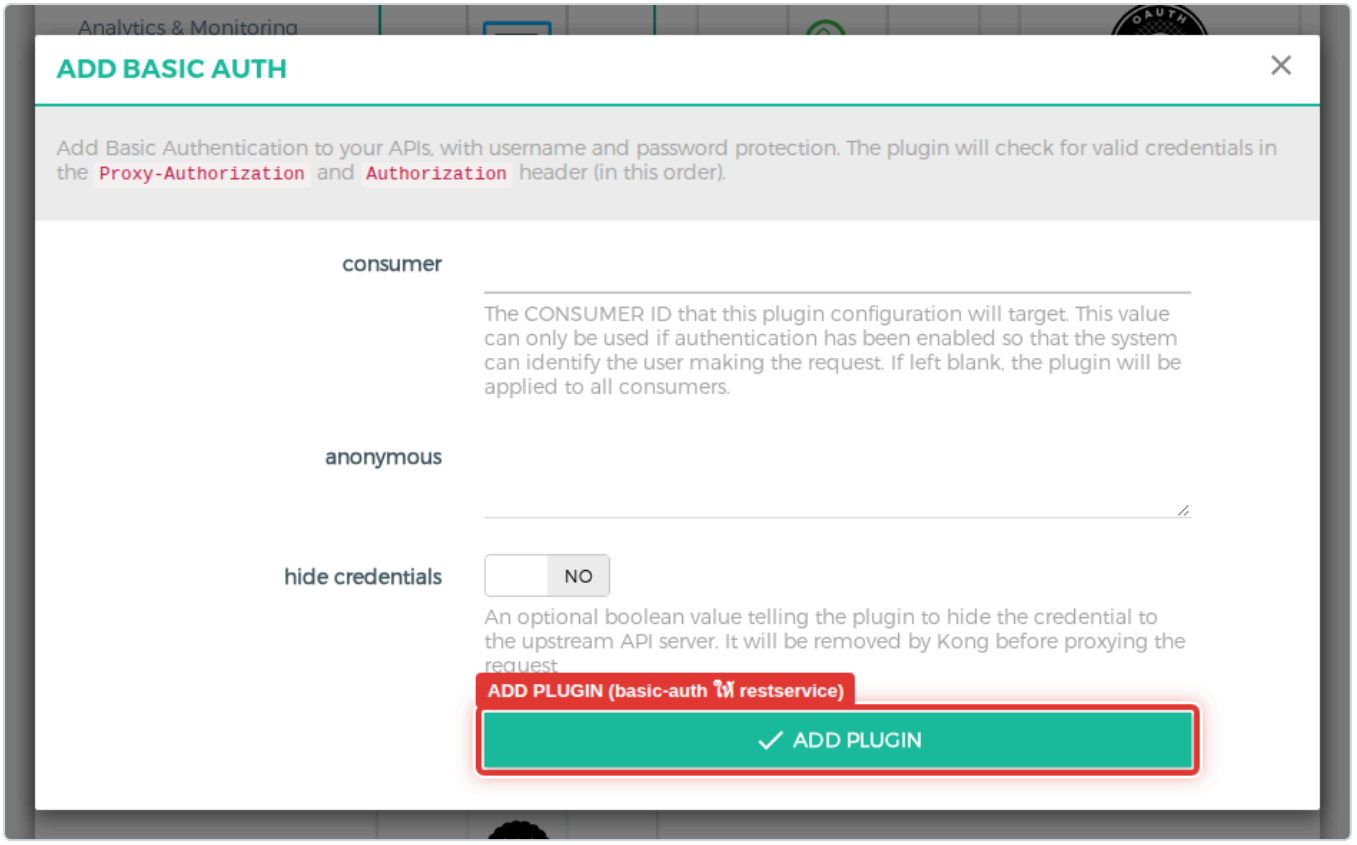
Credentials → API KEYS: key = bob-secret-key-123



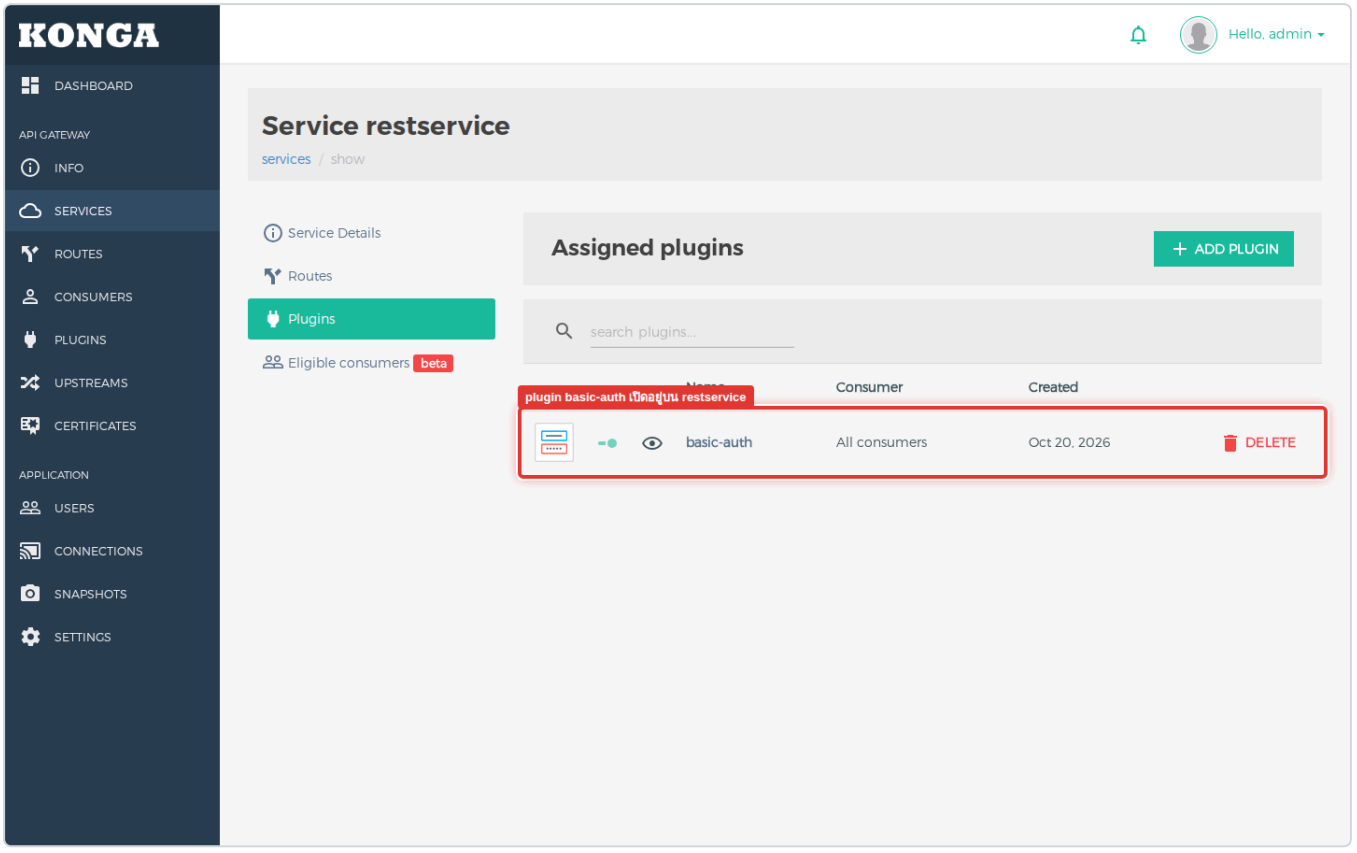
api key ของ Bob



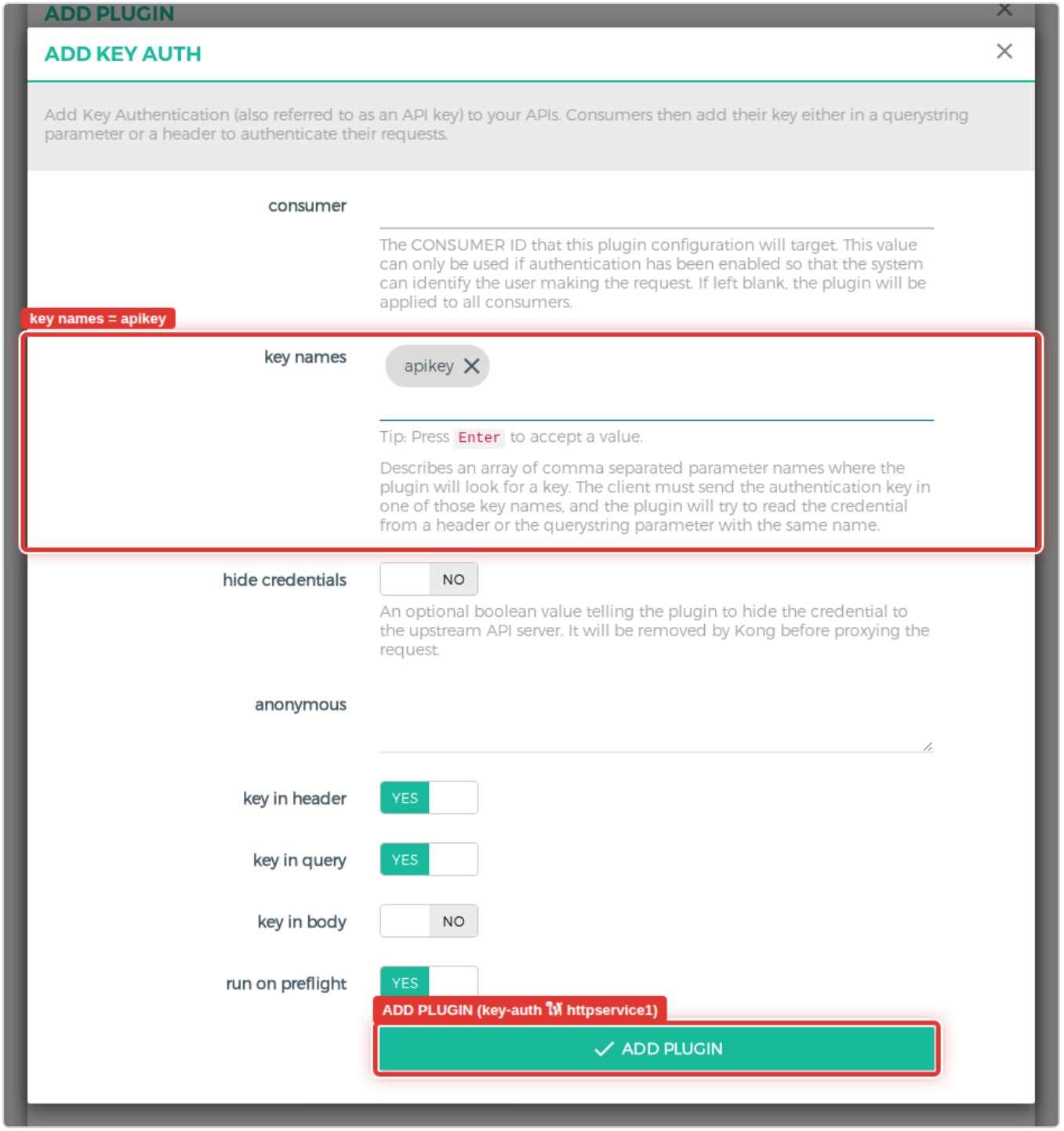
3.2 restservice → Plugins → ADD PLUGIN → Authentication



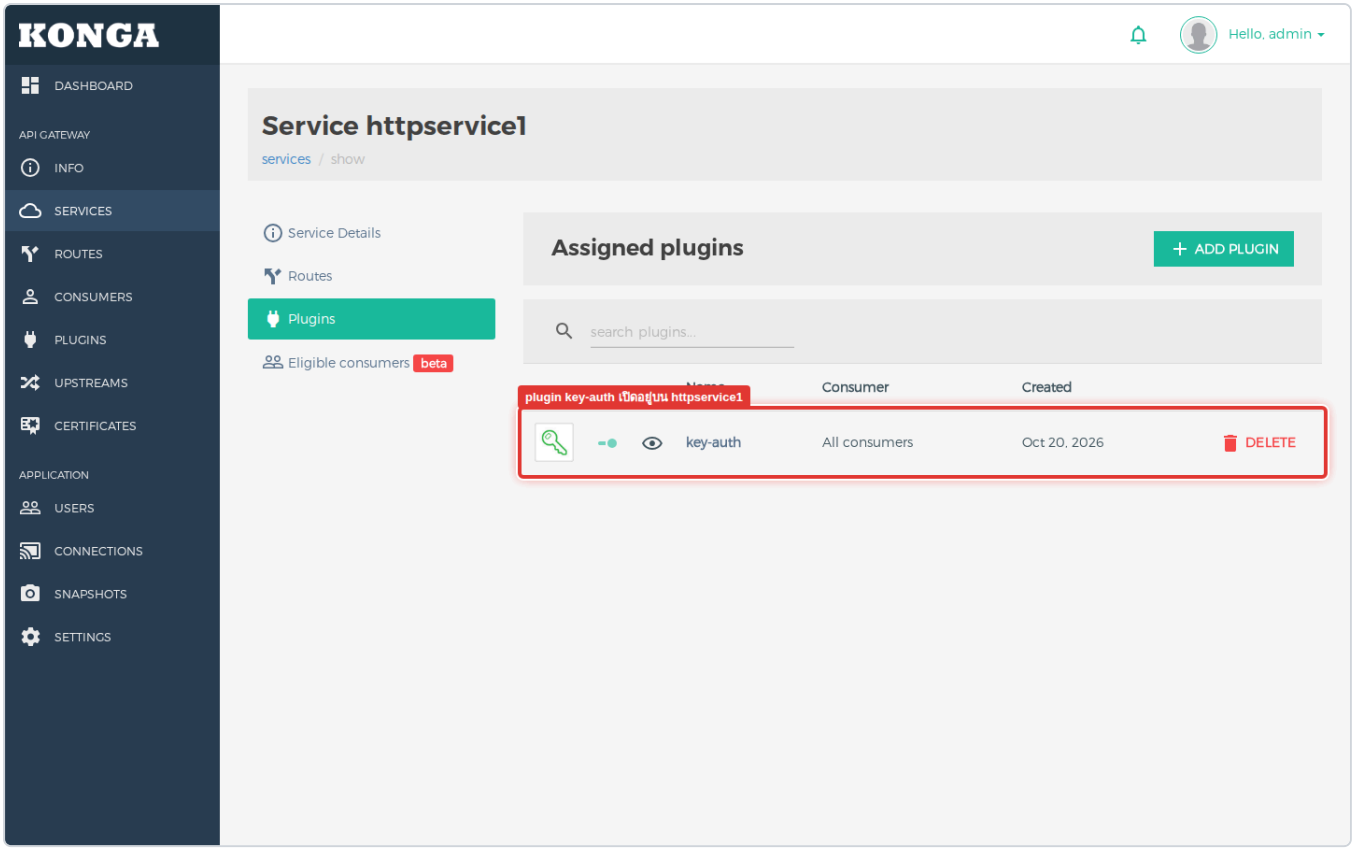
เพิ่ม plugin Basic Auth ให้ restservice



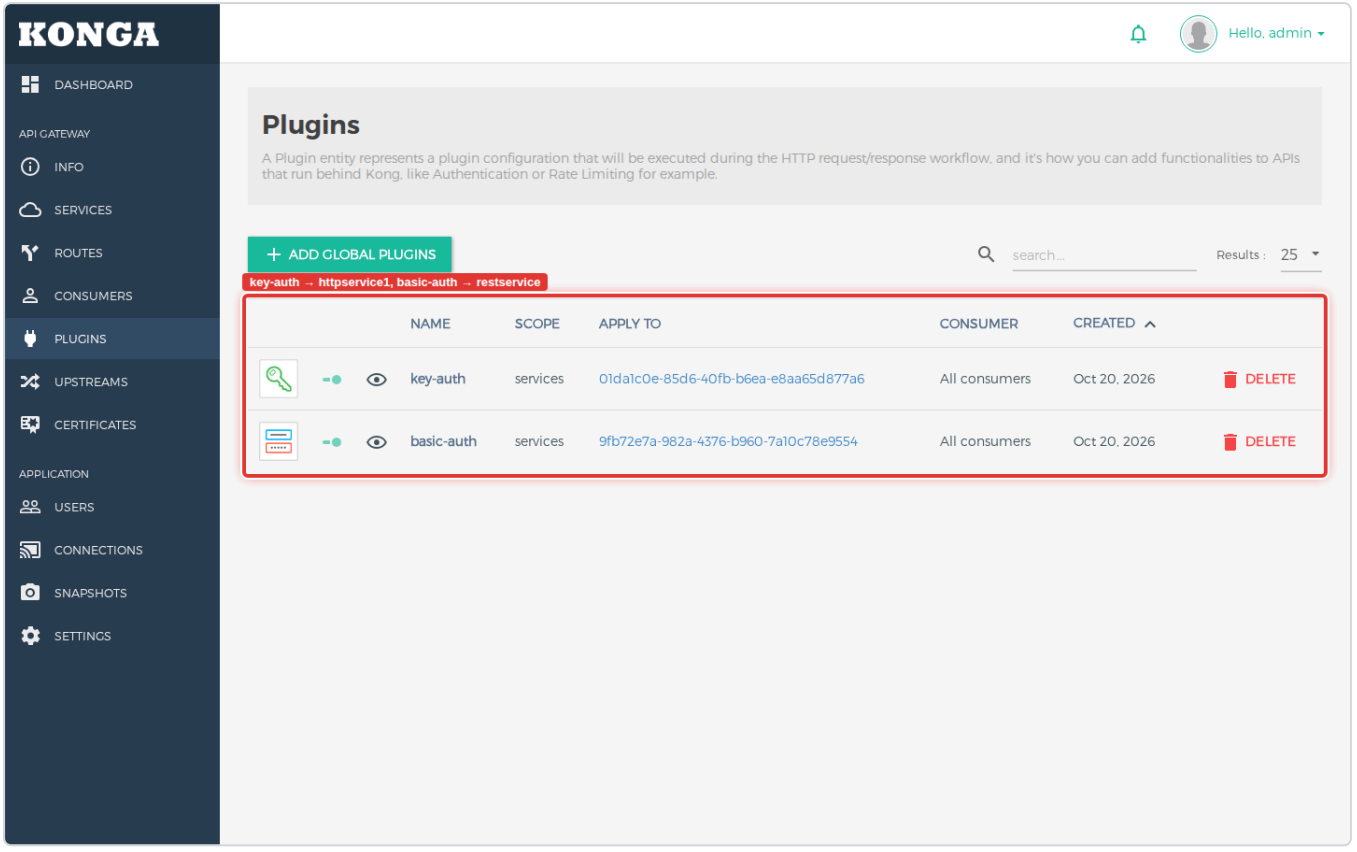
restservice มี plugin basic-auth แล้ว



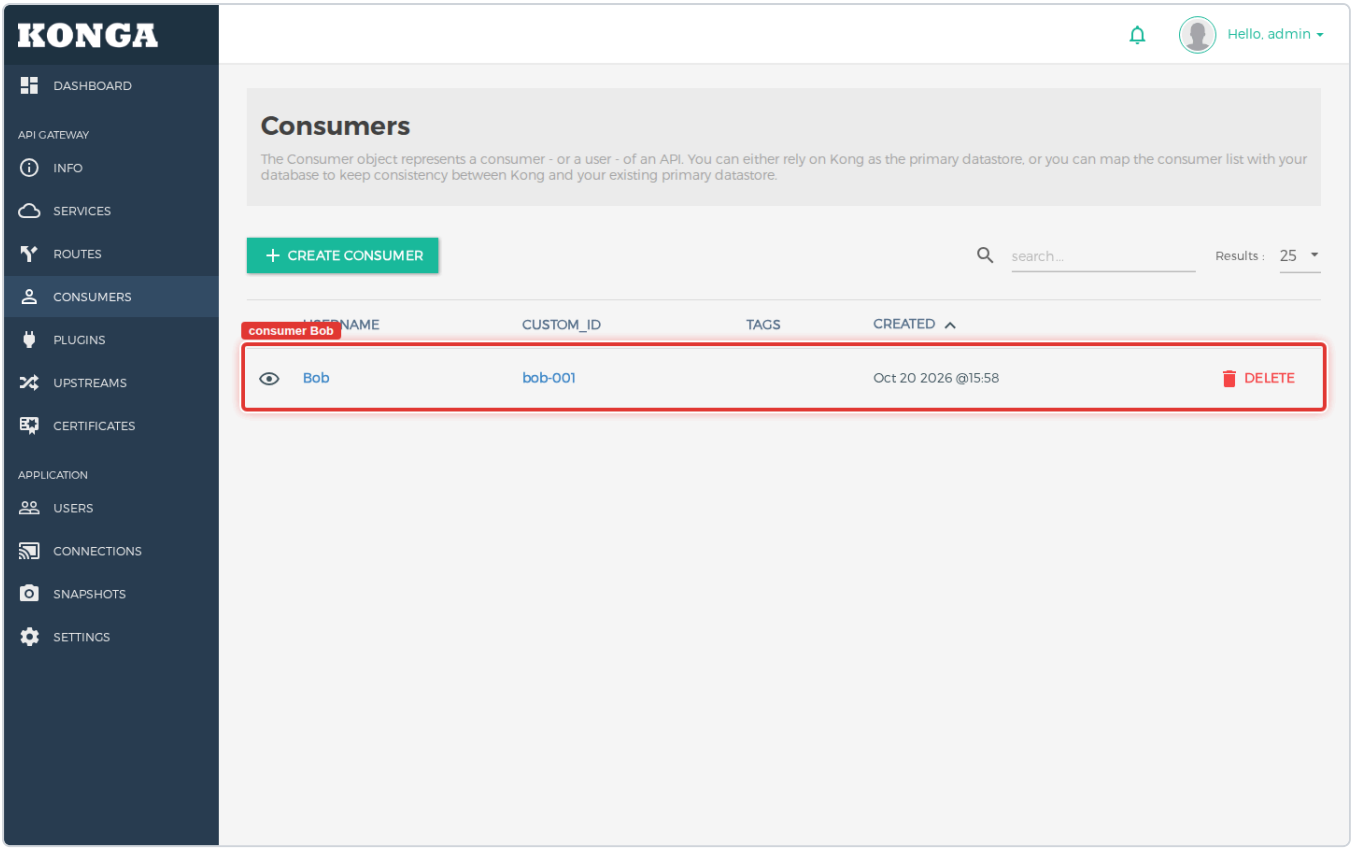
Step 8: เพิ่ม plugin Key Auth ให้ httpservice1 (key name = apikey)



httpservice1 มี plugin key-auth แล้ว



Konga → PLUGINS: key-auth (httpservice1) และ basic-auth (restservice)



Konga → CONSUMERS: Bob

ผลการทดสอบ authentication

```
codespace@codespace_launcher: bash — 20-kong-consumer-credentials

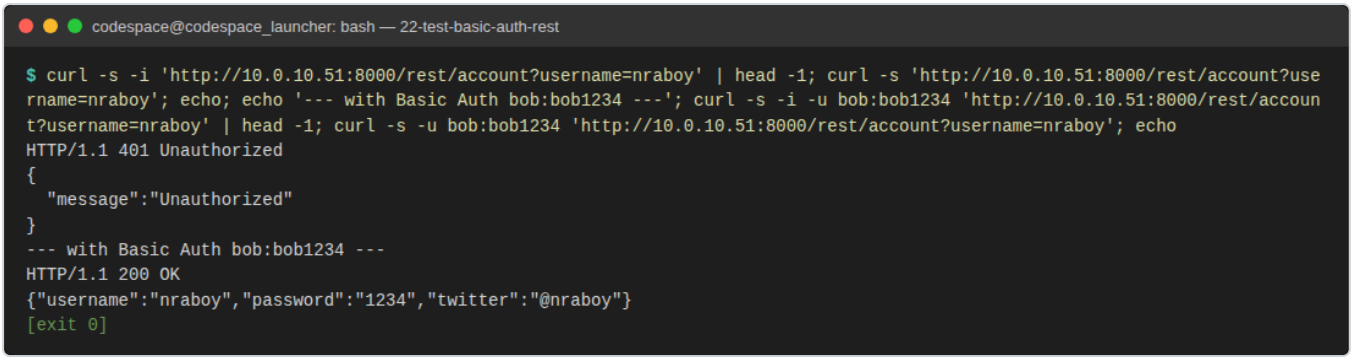
$ curl -s http://localhost:8001/consumers/Bob/basic-auth | python3 -c "import sys,json; [print('basic-auth username =', c['use
rname']) for c in json.load(sys.stdin)['data']]"; curl -s http://localhost:8001/consumers/Bob/key-auth | python3 -c "import sy
s,json; [print('key-auth key =', c['key']) for c in json.load(sys.stdin)['data']]"
basic-auth username = bob
key-auth key = bob-secret-key-123
[exit 0]
```

credentials ของ Bob จาก Kong Admin API

```
codespace@codespace_launcher: bash — 21-kong-plugins

$ curl -s http://localhost:8001/plugins | python3 -c "
import sys,json,urllib.request
svc={s['id']:s['name'] for s in json.load(urllib.request.urlopen('http://localhost:8001/services'))['data']}
[print(p['name'], '-> service', svc.get(p['service']['id'])) for p in json.load(sys.stdin)['data']]"
basic-auth -> service restservice
key-auth -> service httpservice1
[exit 0]
```

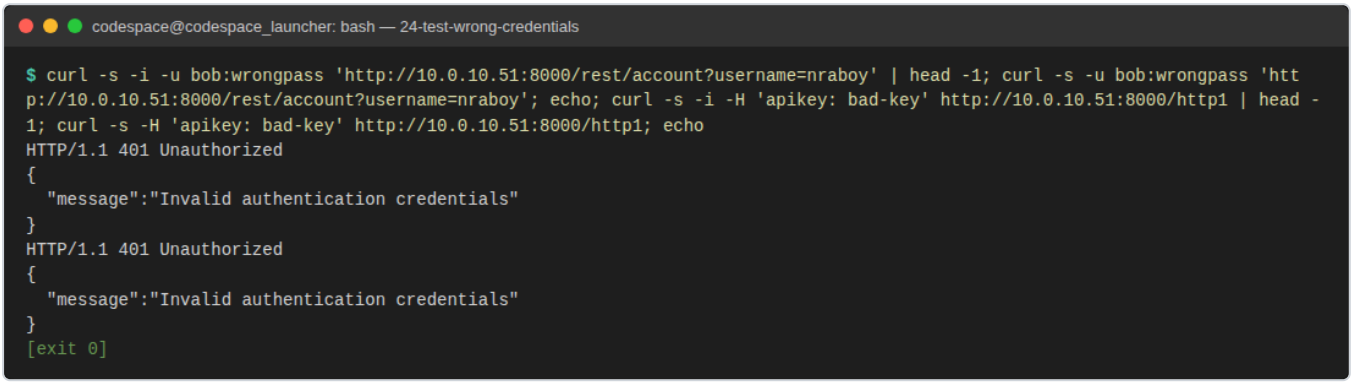
plugins ที่เปิดอยู่และ service ที่ผูก



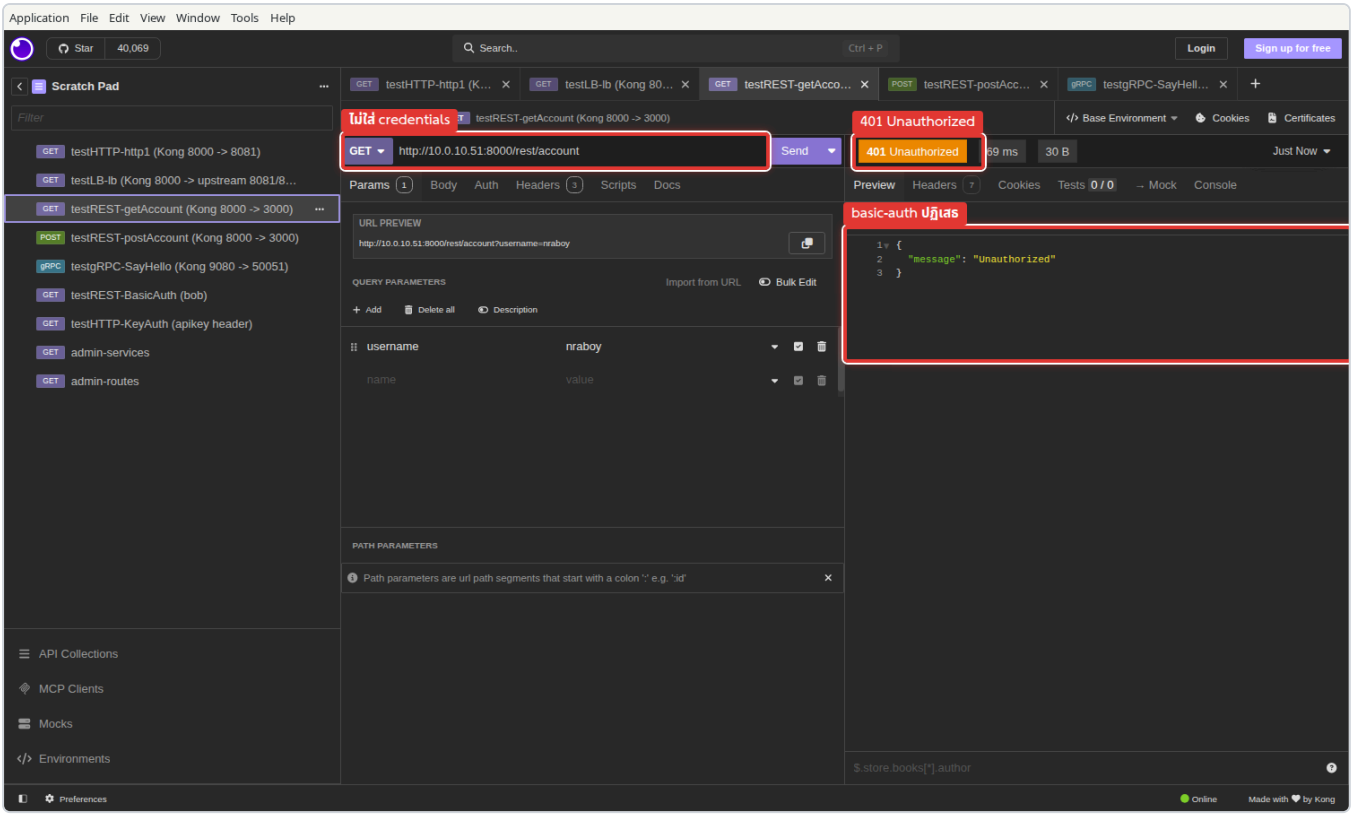
restservice: ไม่ส่ง credentials → 401 Unauthorized, ส่ง -u bob:bob1234 → 200 OK



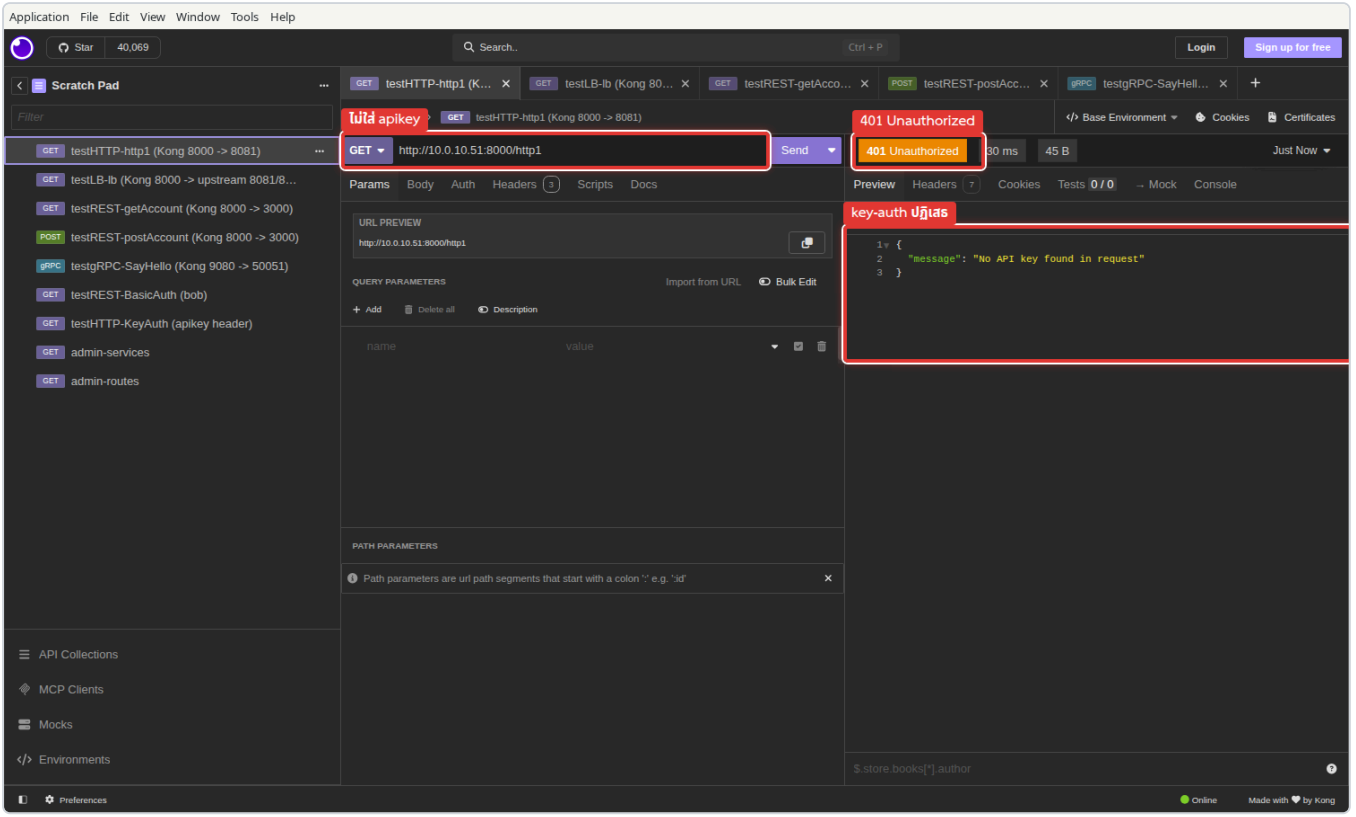
httpservice1: ไม่มี apikey → 401 "No API key found in request", ส่ง apikey ทาง header หรือ query → 200 OK



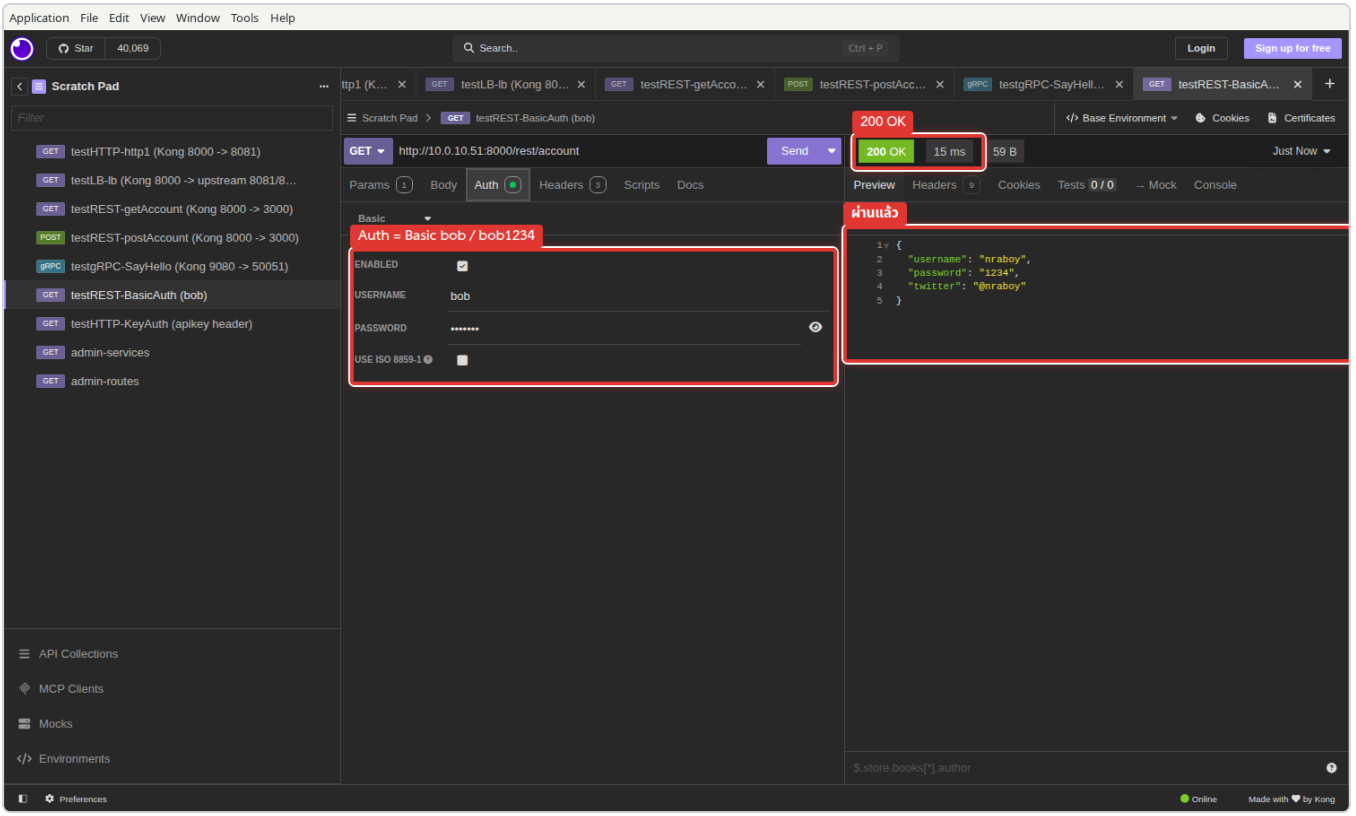
รหัสผ่านผิด / key ผิด → 401 "Invalid authentication credentials"



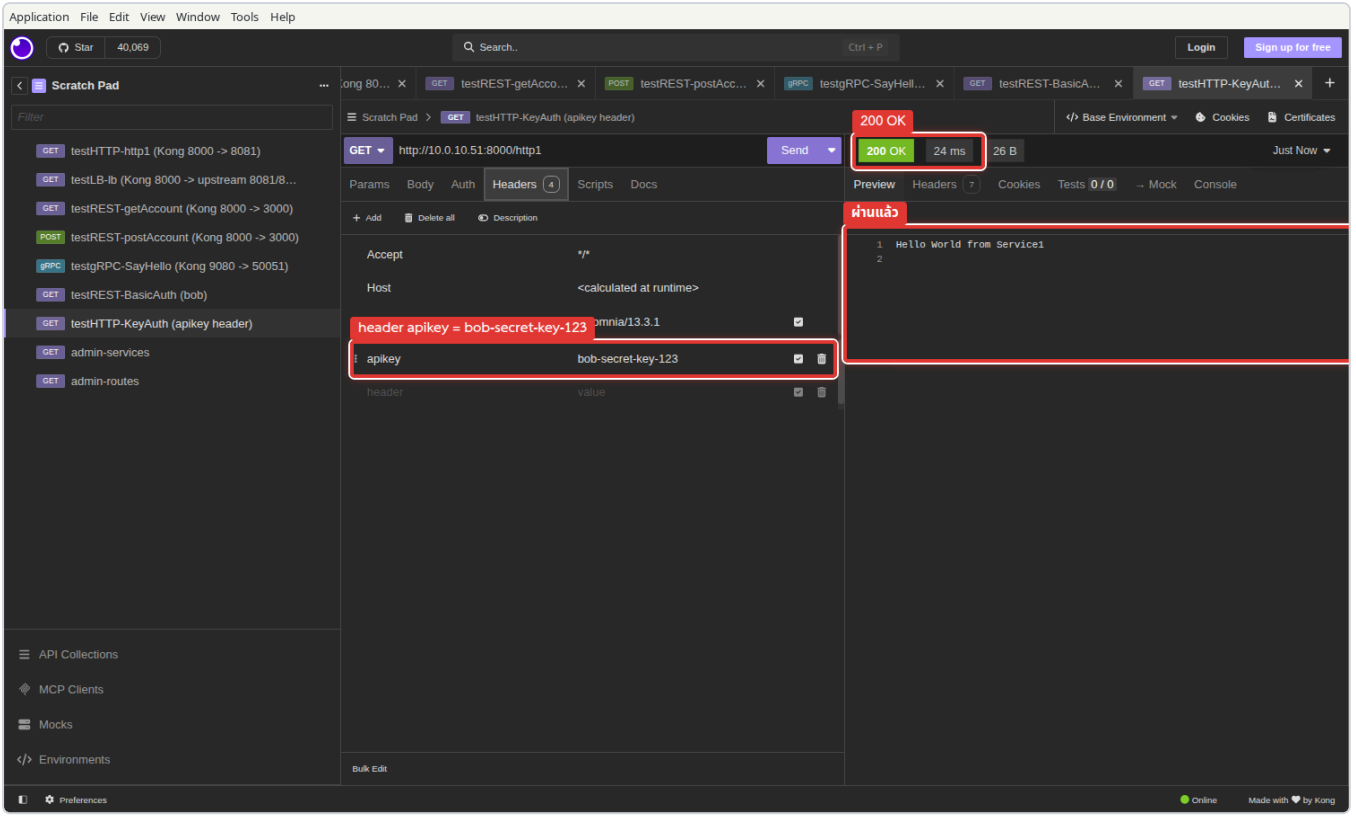
Insomnia: /rest/account **โดยไม่ใส่** auth **หลังเปิด** basic-auth → 401 Unauthorized



Insomnia: /http1 **โดยไม่ใส่** apikey **หลังเปิด** key-auth → 401



Insomnia: Auth = Basic (bob / bob1234) → 200 OK



Insomnia: header apikey: bob-secret-key-123 → 200 OK